



The Open University

M263 Building Blocks of Software

Unit 6

Classes **Vector** and **mString**

Study guide

In terms of their importance, the first three sections of this unit are of about the same weight. Two important collection classes, **Vector of X** and **mString** are introduced, and your study of this unit should focus on becoming familiar with these classes: their form of data and their methods. Section 4 is shorter than the earlier sections. It provides some links to material covered later in the course, and indicates an important application of the material in the first three sections.

There is practical computing work associated with all the material in the unit. This will provide ample opportunity for you to practise, on the Work-Pad, the use of the classes introduced here, and to see their methods in action.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2299 4

Contents

Introduction	4
1 The class Vector	6
1.1 The specification of the class Vector	6
1.2 User code involving the class Vector	11
1.3 while loops	12
2 Vector functions	18
2.1 Deletion, insertion and concatenation	18
2.2 Sorting a sequence	23
2.3 Extending the class Vector	27
3 The class mString	31
3.1 The specification of the class mString	31
3.2 User code involving the class mString	34
3.3 Functions on mString	37
4 Hash tables	42
Summary of Unit 6	46
Solutions to the exercises	46
Index	52

Introduction

Each of the data types **Array of X** and **Stack of X** can be used to hold a collection of one or more elements of a particular type. For instance, we can have an array of elements from **Int** or a stack of objects from the class **Marker**. We will refer to such types as **collection types**. Collections are very important in programming. We often need to store or process several items in the same way, and it is then useful to have the items collected together into a single object. You may recall from Unit 3 that a particular array or stack can only hold a collection of elements belonging to one type. For example, we can have an array of integers (of type **Array of Int**), or an array of characters (of type **Array of Char**). We do not mix elements from **Int** and **Char** in the same array to produce, for instance, an array `[1, 2, 'a', 3]`.

Each of **Array of X** and **Stack of X** is a parameterised type. We obtain a specific instance of such a type when we indicate what **X** is (that is, what type of item is to be stored in the collection). The stored items may come from a primitive type such as **Int** or **Char**, or from a class type such as **Spot**. (We made use of **Stack of Spot** and **Stack of Marker** in Unit 5.)

An array or a stack is a sequence of elements. The difference between these two types lies in the type functions available to manipulate them, and this leads to significant differences in their character and use. Two code fragments are given below. These highlight one major difference in the characteristics of the types **Array** and **Stack**, relating to the size of a variable.

```
var a in Array(of size 3) of Int
a <-- [22,44,33]
```

```
var s in Stack of Int
s <-- PUSH(22,s)
```

For convenience, we will often omit the “**of X**” when referring to these parameterised types in their general form.

The size of an array variable has to be specified in the **var** statement. This size gives the number of elements that the array can hold. The size remains fixed throughout the use of the variable. In particular, we cannot store more items in the array than the number specified as its size in the **var** declaration. In contrast, the size of a stack variable can change. A stack is initialised as an empty stack (which has size 0). Elements can be added to a stack using the function *PUSH*, as in the second fragment above, and the stack will then increase in size. Because the size of a stack can increase as it is used, we say that a stack has **dynamic growth**.

Access to elements provides a second major difference between variables of the two types. An array is indexed. Each element in the array can be accessed by its position in the array. This is done using the type function *AT*. For example, for an array *arr* in the state `[22, 44, 33]`, *AT*(3, *arr*) will return the value 33, the third element in the array. Furthermore, an element in an array can be changed using the type function *PUT*. For example, an assignment *arr* ← *PUT*(3, *arr*, 99) will change the third element in *arr* to 99 (so changing the state from `[22, 44, 33]` to `[22, 44, 99]`). We say that an array can be **randomly accessed**, meaning that we can access any element

in the array. We often use this feature to access all the elements in turn, starting from the first and working through the array elements in order.

In contrast, elements in a stack can be accessed only at one end. The type function *PEEK* can be used to examine the top element. We can change the state of a stack variable, but again only working at one end (the top). We can add an element to the top of a stack using the type function *PUSH* (causing the stack to grow). The type function *POP* deletes the top element (and therefore causes the stack to reduce in size).

In summary, an array has no growth but has easy access to all its elements, and a stack has dynamic growth but convenient access at one end only. If a collection type is needed, the different characteristics of stacks and arrays make it important to choose carefully between them.

In this unit, and in Units 7 and 8, we shall consider several collection classes. Note that these will be class types, as distinct from the primitive types **Stack** and **Array**. Again, we shall constrain an object of each of these new collection classes to hold elements of only one type.

The first collection class that we shall introduce, in Section 1 of this unit, is **Vector of X**. This is a versatile collection class, which combines features of both stacks and arrays. **Vector of X** is a parameterised class. We again do not mix elements of different type within a vector. **X** gives the type of element stored in the vector, and may be any type, such as **Char** or **Spot**.

Like stacks and arrays, an object of type **Vector** again holds its elements in a sequence. The size of the sequence is not fixed, so dynamic growth is possible. The methods of **Vector** also enable random access to the elements in the collection. Many of the methods of **Vector** change the state of the receiver object. We can change the state of an object of this type by adding or removing elements without the need to use assignment statements.

The methods of the class **Vector** make available a wider variety of processes than do either of the types **Stack** or **Array**. There are, however, still a number of tasks on sequences that cannot be performed by direct application of a single method of **Vector**. In Section 2 we consider functions using the class **Vector**, then look at a new class that extends **Vector** by adding further methods.

In Section 3, we introduce a class **mString**. This is a specialised class. An object of this class is a sequence of characters (that is, a string). Methods of this class return a value, and so to write code to change the state of an object of this type we need to use assignment statements. The methods of **mString** enable one to perform a variety of processes on strings, such as extracting part of the string (called a substring), and concatenating two strings.

Section 4 gives a brief introduction to hash codes and hash tables.

Classes similar to those introduced here may be found in many programming languages. You should not expect these M263 classes to be identical to similar classes (possibly with the same name) that you may meet elsewhere.

In this unit (and in Units 7 and 8) the aim is to familiarise you with important forms of collection. The focus is on the form of data and the methods available. We shall not be concerned with how the classes we introduce may be implemented. We shall describe only the specification of each class (not its implementation), then look at user code involving the class.

Remember that all the type functions of **Array** and **Stack** return a value. To change the state of an array or stack variable, we must assign the appropriate value to it.

mString is not a parameterised class.

1 The class Vector

The first collection class we consider is **Vector of X**. This is a useful and versatile class, which has features that enable its objects to behave to some extent both like an array and like a stack. It has the advantages of both random access and dynamic growth. As for **Stack of X** and **Array of X**, the form of data of an object of type **Vector of X** is a sequence of items from **X**. Here, **X** may be any type, including class types. So we might have **Vector of Int** or **Vector of Spot**, for example. We shall not mix items of different types in an object of type **Vector**. Much of our discussion does not depend on the type of item stored, so we shall often refer just to the class **Vector**.

However, note the comment in the Aside on page 11.

Both **Stack** and **Array** have rather few type functions. When working with variables of these types, it is fairly easy to remember what is available to you and what the type functions do. The limited range of type functions makes some tasks difficult to code, though.

The class **Vector** has a larger number of methods. There is a trade-off when working with a large class (that is, a class with a large number of methods). In a small class it is simpler to remember all the methods and what they do, but the code needed to carry out processes on data other than those of the specific methods can become quite involved. In a large class, more tasks can be coded simply, but the increase in the number of methods means that reference to some summary of the methods is often needed.

In this section we shall describe the specification of the class **Vector**. We shall also introduce a new feature of the course code, the **while** loop. This is a form of loop statement different from a **for** loop. A **while** loop is controlled by a condition. Some tasks are much more easily coded using a **while** loop than using a **for** loop.

1.1 The specification of the class Vector

The state of an object of type **Vector of X** is a sequence of items from **X**. The state of a vector will be displayed using the standard sequence notation (as are arrays and stacks). For example, the state of a vector might be given as [11, 22, 33, 44]. In this example, 11 is said to be the **first** element, at the **front** of the vector, and 44 is said to be the **last** element, at the **back** of the vector. The elements are indexed in the same way as the elements in an array. We start at the front with index 1. So in the vector [11, 22, 33, 44]:

11 has index 1; 22 has index 2; 33 has index 3; 44 has index 4.

The default state of the class **Vector** is an empty sequence, so after a statement

```
var myVector in Vector of Int
```

the state of *myVector* is an empty sequence. Unlike **Array**, **Vector** does not require a declaration of the size of a vector object, and there are methods available that allow items to be added to or removed from a vector.

For convenience, we shall often refer to an object of type **Vector of X** simply as a vector.

Sometimes one may want to set the state of a vector to a particular value. For a variable of primitive type we can do this using an assignment, if we have a notation for literal values available. For example, we can declare a variable of type **Array of Int**, and set its state to [1, 2, 3], as in the code fragment below.

```
var myArr (of size 3) in Array of Int
myArr <-- [1,2,3]
```

There is no direct equivalent of a literal notation for the class **Vector**, but we have a method that achieves the same effect. The statements

```
var myVector in Vector of Int
myVector.setVector(11,22,33,44)
```

will declare an object *myVector* of type **Vector**, then set the state of *myVector* to [11, 22, 33, 44].

The method *setVector* is typically used immediately after a **var** statement, to set the vector to a particular value. (The same effect could be achieved by use of other methods, but these add elements to a vector one at a time so it would be a chore to build up a vector in a particular required state.)

The method *setVector* has a parameter. That parameter is a list of elements, each of the required type, separated by commas. The parameter is in essence a sequence of elements, but we do not enclose this sequence in square brackets. The resulting state of the receiver will be shown using square brackets. So all we need to do to show the state after a call of *setVector* is to put square brackets round the list of elements in the input parameter (without changing the order of the list).

A specification of the class **Vector** is given below.

Some programming languages have a form of statement (called a **constructor**) that combines the declaration of an object and the setting of the state of this object to a particular value.

Vector of X	Class specification
<i>States</i>	The state of an object is a sequence of items from X .
<i>State notation</i>	The state is shown in the usual sequence notation.
<i>Default</i>	The empty sequence, [].
<i>Methods</i>	
<i>setVector</i> (list of X)	state change
<i>size</i> ()	return in Int
<i>addFirst</i> (<i>x</i> in X)	state change
<i>addLast</i> (<i>x</i> in X)	state change
<i>setAt</i> (<i>n</i> in Int , <i>x</i> in X)	state change
<i>removeFirst</i> ()	state change
<i>removeLast</i> ()	state change
<i>isEmpty</i> ()	return in Bool
<i>contains</i> (<i>x</i> in X)	return in Bool
<i>first</i> ()	return in X
<i>last</i> ()	return in X
<i>at</i> (<i>n</i> in Int)	return in X

That is, the items in the vector are given between square brackets, and separated by commas.

This class has more methods than classes you have met so far. Each method carries out a single action, and the method names are to an extent self-explanatory. There are a number of methods that change the state of the receiver, and also a number of methods that return a value associated with the current state of the receiver. We now give specifications of the methods.

method *setVector(list of X)* **state change**

pre true.

post The state of the receiver object becomes the sequence given in *list*.

The parameter *list* represents a sequence of values of type **X**. For example, for a vector *v* of type **Vector of Int**, we might set its state to [4, 5, 7] by the statement *v.setVector(4, 5, 7)*, or to [12, 5, 2, 9, 5] by *v.setVector(12, 5, 2, 9, 5)*, or even to the empty sequence by *v.setVector()*.

If the receiver object contains any data, then *setVector* will discard that data and replace it with the parameter *list* (as does assignment to a literal value for a variable of primitive type). However, we shall typically use *setVector* directly after a **var** statement, when the receiver is an empty sequence.

A note of caution is needed here. The parameter *list* of *setVector* must be provided as a list of specific values from **X**. An array (or a stack or a vector) of integers will not be regarded as a list of integers in this context. For example, suppose that the variable *anArr* is of type **Array of Int**. If *anArr* is entered in place of *list*, then it will be regarded as a single item, suitable for storage in a vector object whose type is **Vector of Array of Int**. After a statement *aVect.setVector(anArr)*, the vector *aVect* will hold a single item — the array.

The method *size* returns the number of elements stored in a vector.

method *size()* **return in Int**

pre true.

post The returned value is the number of elements in the receiver.

There are a number of methods that change the state of the receiving vector. The first two add an element to the vector.

method *addFirst(x in X)* **state change**

pre true.

post Changes the state of the receiver by adding the item *x* to the front.

method *addLast(x in X)* **state change**

pre true.

post Changes the state of the receiver by adding the item *x* to the back.

The method *setAt* replaces the item at a specified location in a vector with a new item supplied as a parameter. The precondition ensures that *n* is a valid position in the vector.

method *setAt(n in Int, x in X)* **state change**

pre $1 \leq n$ and $n \leq$ the size of the receiver.

post Changes the state of the receiver by placing the element *x* in the *n*th position (replacing whatever was there before).

Each of the next two methods deletes an item from a vector.

method *removeFirst()* **state change**

pre The receiver is not empty.

post Changes the state of the receiver by deleting its first element.

method *removeLast*() **state change**

pre The receiver is not empty.

post Changes the state of the receiver by deleting its last element.

The next two methods return Boolean values. The method *isEmpty* tests whether or not a vector is an empty sequence; the method *contains* tests whether or not a vector contains a particular element.

method *isEmpty*() **return in Bool**

pre **true**.

post The returned value is **true** if the receiver is empty, and is **false** otherwise.

method *contains*(*x* in X) **return in Bool**

pre **true**.

post The returned value is **true** if the item *x* is an element of the receiver, and is **false** otherwise.

The remaining methods return particular elements from a vector.

method *first*() **return in X**

pre The receiver is not empty.

post The returned value is the first element in the receiver.

method *last*() **return in X**

pre The receiver is not empty.

post The returned value is the last element in the receiver.

method *at*(*n* in Int) **return in X**

pre $1 \leq n$ and $n \leq$ the size of the receiver.

post The returned value is the element in the *n*th position in the receiver.

As for *setAt*, the precondition of *at* ensures that *n* is a valid position in the receiving vector.

The methods of the class **Vector** enable a vector both to be randomly accessed and to have dynamic growth. We can access or change any element of the vector, by reference to its index position. Elements can be added or deleted at either end of a vector.

We now look at some examples of the effect of the methods. We look first at methods that return a value, then at those that change the state.

Activity 1.1

Suppose that an object *v* of type **Vector of Int** is in the state [8, 5, 23, 14, 1]. What value will be returned by each of the following?

- (a) *v.size*()
- (b) *v.isEmpty*()
- (c) *v.contains*(23)
- (d) *v.first*()
- (e) *v.last*()
- (f) *v.at*(4)

Discussion

- (a) There are five items in the sequence [8, 5, 23, 14, 1], so 5 will be returned.
- (b) The size of *v* is not 0, so **false** will be returned.
- (c) The number 23 does appear in [8, 5, 23, 14, 1], so **true** will be returned.
- (d) The method *first* returns the first item in *v* (shown at the left-hand end). So 8 will be returned.
- (e) The method *last* returns the last item in *v* (shown at the right-hand end). So 1 will be returned.
- (f) The method call *at*(4) returns the element in *v* with index 4. The index is counted from the left-hand end, starting at 1, so 14 will be returned.

Activity 1.2

Trace the state of the vectors *u* and *v* as the code fragment below is executed.

```
var u in Vector of Int
var v in Vector of Int
u.setVector(86,35,23,14,11)           //(1)
v <-- u.clone()                       //(2)
u.removeFirst()                       //(3)
v.removeLast()                        //(4)
v.addFirst(77)                        //(5)
u.addLast(99)                         //(6)
v.setAt(3,44)                         //(7)
```

Note that *clone* is available to **Vector** as it is to all M263 classes.

Discussion A trace table is given below.

Comment	<i>u</i>	<i>v</i>
Before (1), <i>u</i> and <i>v</i> are in the default state.	[]	[]
(1) sets the vector <i>u</i> to the specified state.	[86, 35, 23, 14, 11]	[]
(2) sets the state of <i>v</i> to the current state of <i>u</i> .	[86, 35, 23, 14, 11]	[86, 35, 23, 14, 11]
(3) deletes the first item from <i>u</i> .	[35, 23, 14, 11]	[86, 35, 23, 14, 11]
(4) deletes the last item from <i>v</i> .	[35, 23, 14, 11]	[86, 35, 23, 14]
(5) adds the integer 77 at the front of <i>v</i> .	[35, 23, 14, 11]	[77, 86, 35, 23, 14]
(6) adds the integer 99 to the end of <i>u</i> .	[35, 23, 14, 11, 99]	[77, 86, 35, 23, 14]
(7) replaces the item in <i>v</i> with index 3 by 44.	[35, 23, 14, 11, 99]	[77, 86, 44, 23, 14]

Activity 1.3

Suppose that *u* is an object of type **Vector of Int** in the state [11, 22, 33]. Is *u.removeLast().isEmpty()* a valid code statement?

Discussion The answer is “No”. The method *removeLast* changes the state of the receiver. So the effect of *u.removeLast()* is to change the state of *u* (by deleting its last element). The method *removeLast* does not return a value. So *u.removeLast()* does not represent an object to which the method *isEmpty* can be applied.

Aside

Some programming languages, such as Java, have sophisticated features associated with typing. In particular, a technique called **casting** allows one to move values between different types in a subtle way. We have omitted discussion of such features from this course to avoid complications. In many programming languages, every class is an extension of some basic class (often called **Object**). Indeed, this is the case in the WorkPad, where every class extends **Base**. One can mix objects of different types within a vector, so long as that vector is regarded as containing elements of type **Base**. An element extracted from such a vector is of type **Base**, but if casting were available then it could be used to transfer the data in such an element to an object of a more structured type.

End of Aside

1.2 User code involving the class Vector

We now consider some examples of user code involving the class **Vector**.

Activity 1.4

Give an implementation of the function *ADDCAPS* whose specification is given below. How might you test your implementation?

function *ADDCAPS*(*u* in **Vector of Char**) **return** in **Vector of Char**

pre All elements of the vector *u* are lower-case letter characters.

post The returned value is a vector whose size is double that of *u*. This starts with a copy of *u*, which is followed by the upper-case letter characters corresponding to the lower-case letters in *u* (in the same order).

Discussion An implementation is given below. A variable *v* of type **Vector of Char** is set to a copy of *u*. We then go through each character in *u* in turn (starting at the front), using an index *i*. We extract the *i*th character in *u*, find its upper-case equivalent, then add this at the end of *v*.

```
function ADDCAPS(u)
{
    // impAddCaps
    var v in Vector of Char
    var c in Char
    var i in Int
    v <-- u.clone()
    for (i <-- 1 to u.size())
    {
        c <-- u.at(i)           // the character with index i in u
        c <-- CHR(ASC(c) - 32) // (1) its upper-case equivalent
        v.addLast(c)           // add this to the end of v
    }
    return v
}
```

We noted in Unit 3, p16, that the expression in (1) gives the upper-case equivalent of a lower-case letter character.

To test the implementation *impAddCaps*, we need to find the value returned by the implementation for an input that satisfies the precondition, and compare this against the value that should be returned. For example, we might try ['a','b'] as a test input. The correct output for this input is determined

It is usually advisable to test several different inputs.

by reference to the specification, and is ['a','b','A','B']. One could perform the test by tracing the code in *impAddCaps* by hand. Or one could use the WorkPad, by examining the state of *v* after the following user code.

```
var u in Vector of Char
var v in Vector of Char
u.setVector('a','b')
v <-- ADDCAPS(u)
```

A vector can hold items of any type, such as integers, characters or spots. It can even hold other vectors. For example, a variable of type **Vector of Vector of Int** will be a vector, each of whose elements is itself a vector of integers.

EXAMPLE 1.1

After the code fragment below is executed, what will be the state of *v*? What value would be returned by each of: *v.size()*; *v.at(2).at(1)*?

```
var a in Vector of Int
var b in Vector of Int
var v in Vector of Vector of Int
a.setVector(1,2,3)
b.setVector(5,6)
v.setVector(a,b)
```

The state of *v* after the fragment has been executed can be written as below:

```
[[1, 2, 3], [5, 6]]
```

Note that this is a vector of vectors. Each element of the vector *v* is itself a vector. There are two items in the vector *v*, so *v.size()* will return the value 2. (The items in *v* are themselves vectors, but we are not concerned here with what these two vectors contain.)

v.at(2) will return the second element of *v*. This is a vector whose state is [5,6]. The value returned by *v.at(2).at(1)* will then be the first element in the vector [5,6], that is, the integer 5.

1.3 while loops

In this subsection we shall introduce a new form of statement in the course code. This is a **while** loop, which repeats the body of the loop so long as a specified condition is **true**. This contrasts with a **for** loop, which repeats the body of the loop for a specified range of values of the loop variable.

A specification of a function *SQUARES* is given below. Consider for a moment whether you could implement *SQUARES* using a **for** loop.

function *SQUARES*(*target* in Int) return in Vector of Int

pre *target* > 1.

post The returned value is a vector giving (in increasing order) the squares of the integers from 1 to *n*, where $(n + 1)^2$ is the first squared value that is greater than the number *target*.

For example, if *target* is 20 then the returned value is the vector [1, 4, 9, 16] (since the next square number is 25, which is greater than 20).

The number of items in the vector returned by *SQUARES* is not easily calculated. So it is not easy to use a **for** loop to implement this function. We would like to calculate each squared integer in turn: $1^2 = 1$, $2^2 = 4$, and so on, adding each to the vector to be returned, and stop when the next square value we calculate is too large. To implement this strategy, we need the new form of loop statement.

Code to implement the function *SQUARES* is given below. The **while** loop in *impSquares* is controlled by the condition $n \times n \leq \text{target}$. The body of the loop will continue to execute so long as this Boolean valued condition is **true**. See if you can trace this code when *target* is 10.

Recall that n^2 is $n \times n$.

```
function SQUARES(target)
{
    // impSquares
    var v in Vector of Int
    var n in Int
    n <-- 1
    while (n*n <= target)
    {
        v.addLast(n*n)
        n <-- n + 1
    }
    return v
}
```

A trace of *impSquares* when *target* is 10 is given below. The last column in the table gives the value of the **while** condition. This condition is checked before each execution of the loop body. If it is **true**, then the loop body is repeated. If the **while** condition is **false**, then execution of the loop stops.

Trace with *target* = 10.

Comment	<i>v</i>	<i>n</i>	$n \times n \leq \text{target}$
before the loop	[]	1	
check while condition	[]	1	$1 \times 1 \leq 10$ is true
in loop	[1]	1	
	[1]	2	
check while condition	[1]	2	$2 \times 2 \leq 10$ is true
in loop	[1,4]	2	
	[1,4]	3	
check while condition	[1,4]	3	$3 \times 3 \leq 10$ is true
in loop	[1,4,9]	3	
	[1,4,9]	4	
check while condition	[1,4,9]	4	$4 \times 4 \leq 10$ is false

Since the **while** condition is now **false**, the loop ceases execution. The final statement in *impSquares* then returns the current value of *v*, which is [1, 4, 9]. This satisfies the postcondition of *SQUARES* (9 is less than the value of *target*; the next square, 16, is greater than *target*) and so is correct.

In general, a **while** loop consists of a statement

while (*condition*)

followed by a block of statements, enclosed between { and }, which form the **body** of the loop. As with a **for** loop, the statements forming the body of

the **while** loop are to be repeated. The difference lies in how this repetition is controlled.

In a **while** loop, *condition* is Boolean valued. Before the loop is entered, and prior to each subsequent repetition, *condition* is evaluated using the current states of the variables. If *condition* evaluates to **true**, then the statements forming the body of the loop are performed. If *condition* evaluates to **false**, then execution of the loop body stops. The next statement to be executed is that following the **while** loop (i.e. the statement after the closing bracket following the loop body), as with a **for** loop.

We refer to the Boolean valued *condition* as the **while condition**.

Activity 1.5

With $target = 14$, trace the code below and give the value returned.

```
function ACT(target)
{
  var i in Int
  var n in Int
  i <-- 3
  n <-- 0
  while (i <= target)
  {
    i <-- i + 3
    n <-- n + 1
  }
  return n
}
```

Discussion A trace with $target = 14$ is given below.

Comment	i	n	$i \leq target$
before loop	3	0	$3 \leq 14$ is true
enter loop	6	1	$6 \leq 14$ is true
	9	2	$9 \leq 14$ is true
	12	3	$12 \leq 14$ is true
	15	4	$15 \leq 14$ is false
end loop			

The final statement returns the value of n , which is 4.

This trace is a little more abbreviated than the previous one. This is the form we shall usually use when tracing a **while** loop.

Look back at the code in *impSquares*. Here are some points to note about that, which relate to code using a **while** loop (rather than a **for** loop).

1. The key variable controlling whether or not the loop body is repeated is n . The **while** condition is $n \times n \leq target$. If $target$ is 100, for example, then the loop continues so long as n^2 is less than or equal to 100, and the loop stops when n^2 becomes greater than 100.
2. We initialise the variable n (as 1) before the loop.
3. The variable n is changed (increased by 1) inside the body of the loop. This is achieved by the statement $n \leftarrow n + 1$. As a result, the **while** condition must eventually evaluate to **false**, causing the loop to stop.

In a **for** loop, the processes in 2 and 3 are automatic: the loop variable is given an initial value, and is increased (by 1) after each execution of the loop body, without any need for explicit statements to do this.

One of the most common errors when coding a **while** loop is not to initialise the variable that governs the **while** condition. This must be done before the loop (outside the loop body), and so before the **while** condition is first evaluated. Another common mistake is not to ensure that the variable that governs the **while** condition actually changes inside the loop; this is essential to ensure that the loop will eventually stop executing.

Activity 1.6

Look at the code in Activity 1.5.

- (a) (i) What value would be returned if *target* is negative?
- (ii) What variable controls the **while** loop in that code? Where is that variable initialised? Is its value changed within the body of the loop?
- (iii) Does the code ensure that, whatever integer value of *target* is input, the **while** loop will eventually stop executing?
- (b) Suppose that the **while** condition is changed to $NOT(i == target)$. Would the **while** loop then always stop executing, whatever value of *target* is input?

Discussion

- (a) (i) Before the loop, *i* is set to 3. If *target* is negative, then the **while** condition $i \leq target$ will be **false** when it is first met (since *i* is 3 and *target* is negative). So the loop body will never be executed. The variable *n* will remain as 0, and so the value returned will be 0.
- (ii) The **while** condition is $i \leq target$, so the variable *i* controls execution of the loop.
The variable *i* is initialised before the loop, in the statement $i \leftarrow 3$.
The value of *i* is changed within the body of loop, by the statement $i \leftarrow i + 3$.
- (iii) The value of *i* is increased (by 3) each time the body of the loop is executed. Since *target* is a fixed value, *i* must eventually become larger than *target*. The **while** loop will then stop executing. (The short answer to the question is “Yes”.)
- (b) If the **while** condition were $NOT(i == target)$, then the **while** loop would stop executing when *i* is equal to *target*. There is no guarantee that this will ever happen, though. As the loop body is executed repeatedly, *i* will take the values: 3, 6, 9, 12, 15, and so on. With *target* = 14 (as in Activity 1.5), the condition $i == 14$ would never become **true**. So the **while** condition $NOT(i == 14)$ would never become **false**, and the **while** loop would execute indefinitely.

Activity 1.6(b) illustrates that care is needed when using a **while** loop. One needs to ensure that the **while** condition must eventually become **false**. Otherwise the loop body will execute indefinitely.

We now have two different types of loop available for use, the **for** loop and the **while** loop. How might you decide which type of loop is most appropriate to use in a particular problem? The first thing to consider is

whether you know the number of times the loop should be repeated. If the loop body should be repeated a known number of times, then a **for** loop is likely to be appropriate. If, for example, you want to add up all the numbers in a sequence, then one can use a **for** loop, with the length of the sequence setting the number of repetitions. However, if the number of times that the loop body should be repeated cannot be determined, then a **while** loop is likely to be appropriate.

Activity 1.7

A specification of a function *FIRSTAT* is given below. Give an implementation of *FIRSTAT*. (Use a **while** loop.)

function *FIRSTAT*(*v* in **Vector of Char**, *x* in **Char**) **return** in **Int**

pre *v* contains at least one copy of *x*.

post The returned value is the index of the first occurrence of *x* in *v*.

For example, *FIRSTAT*(['t','h','e','r','a','c','e','s'],'e') is 3. The character 'e' occurs twice in the vector ['t','h','e','r','a','c','e','s'], with index 3 and with index 7, but the first occurrence is with index 3, so this is the value returned.

Discussion We can implement *FIRSTAT* as follows.

Set an integer variable (say *i*) to the initial value 1.

Keep increasing *i* by 1, until the element of *v* with index *i* is equal to *x*.

Stop at this point, and return the value of *i*.

We can ensure that the repetition stops at the correct point by using a **while** loop, with condition *NOT*(*v.at(i) == x*). This loop will stop when *v.at(i)* is equal to *x*, as required. (The precondition of *FIRSTAT* ensures that this will happen sometime.)

An implementation is given below.

```
function FIRSTAT(v,x)
{
  var i in Int
  i <-- 1
  while (NOT(v.at(i) == x))
  {
    i <-- i + 1
  }
  return i
}
```

We will not discuss the implementation of the class **Vector**. Most object-oriented programming languages will have a class **Vector** similar to the M263 class specified above, but do not expect such classes to be identical to the M263 class.

Objectives for Section 1

After studying this section you should be able to do the following.

- ❑ Recognise and use the terminology associated with the class **Vector**, the names of its methods, and terms such as front, back, first and last specifying locations within the sequence representing a vector.
- ❑ Recognise and use the terminology: collection type, random access, dynamic growth, **while** loop, **while** condition.
- ❑ Understand the characteristics of the class **Vector of X** and use the methods of this class in code.
- ❑ Interpret and use the notation for a **while** loop in the course code.
- ❑ Be aware that a **while** loop may execute indefinitely if incorrectly coded.

Exercises on Section 1

Exercise 1.1

- (a) Trace the states of the variables as the code fragment below is executed.

```
var v in Vector of Int
var i in Int
var found in Bool
v.setVector(1,99,3,99,5)      //(1)
i <-- 0
found <-- false
while (NOT(found))
{
  i <-- i + 1
  if (v.at(i) == 99) then
  {
    found <-- true
  }
}
v.setAt(i,100)
```

- (b) Suppose that statement (1) is changed to *v.setVector(1, 2, 3, 99, 5)*. What will be the final states of the variables in this case?
- (c) If *v* is set to an inappropriate value in statement (1), then this fragment is invalid. Give an example for which the code is invalid.

Exercise 1.2

A specification is given below of a function *MAX*. Give an implementation of *MAX*.

function *MAX*(*v* in **Vector of Int**) **return** in **Int**

pre *v* contains at least one number.

post The returned value is the largest number in *v*.

For example, $MAX([14, 3, 12, -21, 11]) = 14$.

Exercise 1.3

A specification is given below of a function *FIRSTOVER*. Give an implementation of *FIRSTOVER*. (Use a **while** loop.)

function *FIRSTOVER*(*target* in Int, *v* in Vector of Int) **return** in Int
pre true.

post Starting at the front of the vector *v*, examine each element in turn until an integer (say *n*) is found for which *target* < *n*. The returned value is the index position of *n* in *v*. If there is no element *n* in *v* that satisfies this condition, then the returned value is 0.

For example, *FIRSTOVER*(10, [4, 7, 10, 7, 12, 20, 8]) is 5. This is because: the entry 12 in the vector [4, 7, 10, 7, 12, 20, 8] is greater than the input value 10; none of the entries (4, 7, 10, 7) before 12 in the vector is larger than 10; and the index of 12 in the vector is 5.

2 Vector functions

The class **Vector** provides a range of methods, making it a versatile class for handling sequence-based collections. However, there are a number of processes on sequences that are not available as methods of the M263 class **Vector**. In this section we shall consider how to implement some of these. We shall look first at functions on **Vector**. Later, we shall specify a class **VectorPlus** that extends the class **Vector** to add extra methods.

Some of these may be included in **Vector** classes available in particular programming languages.

This section will both provide practice in using the class **Vector**, and introduce you to some further processes on sequences that are of interest in themselves. The most important of these new processes is sorting, which we consider in Subsection 2.2.

2.1 Deletion, insertion and concatenation

The class **Vector** has methods to delete elements from either end of a vector, but does not have a method to delete an element at any other position in a vector. Consider the following function specification. (The precondition ensures that *n* refers to an actual position in the vector *v*.)

function *REMOVE*(*n* in Int, *v* in Vector of X) **return** in Vector of X

pre $1 \leq n$ and $n \leq$ the size of *v*.

post The returned value is a vector which is identical to *v* except that the element with index *n* in *v* has been removed.

Activity 2.1

Suppose that v is the vector $[8, 9, 7, 6]$. What is $REMOVE(n, v)$ for each of the following values of n ? (a) 0; (b) 1; (c) 3; (d) 6.

Discussion The first thing to note here is that two of the given values of n , those in (a) and (d), do not satisfy the precondition of $REMOVE$. A call of the function $REMOVE$ with the given value of v , and with n equal to either 0 or 6 would be invalid.

- (a) Since $n = 0$ does not satisfy $1 \leq 0$, this value of n does not satisfy the precondition of $REMOVE$.
- (b) The returned value is obtained by deleting from $[8, 9, 7, 6]$ the item at index position 1. So the vector $[9, 7, 6]$ is returned.
- (c) The returned value is obtained by deleting from $[8, 9, 7, 6]$ the item with index position 3. So the vector $[8, 9, 6]$ is returned.
- (d) The vector $[8, 9, 7, 6]$ has size 4. Since $6 \leq 4$ is false, the inputs $v = [8, 9, 7, 6]$ and $n = 6$ do not satisfy the precondition of $REMOVE$.

Remember that $REMOVE$ is a function. As such, it returns a value, and does not change the state of any of its inputs. If, for example, you wanted to use $REMOVE$ to remove the third element of a vector v , you would need to use an assignment statement, $v \leftarrow REMOVE(3, v)$. On its own, a call of $REMOVE(3, v)$ has no effect on the state of v , and $REMOVE(3, v)$ just represents a transient value (returned by the function), that will only be available later if captured by assignment.

Similar comments apply to any function, but these points are rather easy to forget when working with a class many of whose methods are state changers.

An implementation *impRemove* of $REMOVE$ is given below. It uses the following strategy.

Declare a local variable of type **Vector**, *temp*, which will start empty. Working from the front of the input vector, place the items at the first $n - 1$ positions into *temp*. (Place each item at the back of *temp*, so as not to change the order in which the items appear in the vector.)

Omit the item with index n .

Continue with the elements of the input vector from index $n + 1$ to the end.

```
function REMOVE(n,v)
{
    //impRemove
    var temp in Vector of X
    var i in Int
    for (i <-- 1 to n-1)
    {
        temp.addLast(v.at(i))
    }
    /* Omit the item with index n. */
    for (i <-- n+1 to v.size())
    {
        temp.addLast(v.at(i))
    }
    return temp
}
```

Activity 2.2

Trace *impRemove* for the inputs $v = [8, 9, 7, 6]$ and $n = 3$, and verify that the correct value is returned in this case.

Discussion A trace table is given below. Since n is 3, the first **for** loop is executed with i set to 1 then 2. Since the size of v is 4, the second **for** loop is executed “from 4 to 4”; that is, just once, with i set to 4.

Trace with $v = [8, 9, 7, 6]$ and $n = 3$.

Comment	i	$temp$
before first loop	—	[]
first loop	1	[8]
end first loop	2	[8, 9]
second loop	4	[8, 9, 6]

The value returned is the vector $[8, 9, 6]$. This is as given by the specification of *REMOVE* (see Activity 2.1(c)), and so is correct for these inputs.

We could use a trace similar to that in Activity 2.2 to show that, if the state of v is $[8, 9, 7, 6]$, then *impRemove* behaves correctly for any value of n between 1 and 4.

Now the specification of the function *REMOVE* contains a precondition, and it is the responsibility of a user of *REMOVE* to check that their input values satisfy this precondition. The code in *impRemove* does not need to check that n lies in the range given in the precondition, and does not do so. If a user tries to call a function in circumstances that break a precondition, then they are writing invalid code, and the effect of this will be unpredictable.

This comment does not only relate to *REMOVE*, of course. It applies to any function with a precondition.

As it happens, the code in *impRemove* will return a value if v is $[8, 9, 7, 6]$ and n is 0, even though these input values do not satisfy the precondition. If n is 0, then the first **for** loop will not be executed at all, since $n - 1$ will be less than 1. The second **for** loop will be repeated with values of i from 1 to 4 (the size of the given input vector v). In this case, the value returned by *impRemove* will be a vector identical to the input vector v . This result may seem reasonable, since there is no element at position 0 in v to remove.

If v is $[8, 9, 7, 6]$ and n is 5, then the code in *impRemove* will again return a vector identical to v . The items in v will this time be copied to $temp$ by the first **for** loop in *impRemove*, and the second **for** loop will never be executed. However, *impRemove* behaves quite differently with a larger value of n , such as 6. In that case, the first **for** loop will call $v.at(5)$. This breaks the precondition of the method *at*, since the size of v is 4. The code in *impRemove* is then invalid, and its effect cannot be predicted.

When an implementation is called for input values that fail the precondition, anything may happen! You may get a “reasonable” value returned, or you may get nothing. Do not be fooled by the behaviour of *impRemove* when n is 0 into thinking that it will have the effect of returning a vector identical to the input vector for any value of n that breaks the precondition. This is not the case.

Suppose that you want a function similar to *REMOVE*, but without a precondition, whose effect for “out of range” values of n is to return a vector identical to the input vector. This is a different function from *REMOVE*,

and should be given a different identifier. An implementation could be based on *impRemove*, which will be correct in some cases. But it will need changing, because it will not be correct for other inputs.

We look now at inserting an element into a vector. The methods of **Vector** enable us to insert an element either at the front or at the back of a vector, but not at any other position in the vector. Consider the following function specification.

function *INSERT*(*n* in Int, *v* in Vector of X, *elem* in X) return in Vector of X

pre $1 \leq n$ and $(n \leq 1 + \text{the size of } v)$.

post The returned value is a vector whose size is 1 greater than the size of *v*. It is identical to *v*, except that *elem* has been inserted into *v* so that it is the element with index *n* in the returned vector.

Activity 2.3

Suppose that *v* is the vector [8, 9, 7, 6]. What is *INSERT*(*n*, *v*, 99) for each of the following values of *n*? (a) 1; (b) 4; (c) 5.

Discussion The values are:

- (a) [99, 8, 9, 7, 6];
- (b) [8, 9, 7, 99, 6];
- (c) [8, 9, 7, 6, 99].

Activity 2.4

What is the state of each of the variables *u* and *v* after the code fragment below has been executed?

```
var v in Vector of Char
var u in Vector of Char
v.setVector('a','b','c','d')
u <-- INSERT(2,v,'q')
```

Discussion The state of *v* will be ['a','b','c','d'] and the state of *u* will be ['a','q','b','c','d']. (Remember, calling *INSERT*(2, *v*, 'q') has no effect on the state of *v*.)

Activity 2.5

Give an implementation of *INSERT*. (You may find it useful to refer to *impRemove*, and modify that.)

Discussion We can use the following strategy.

Declare a local variable of type **Vector**, say *temp*, which will start empty.

Place into *temp* the items at the first $n - 1$ positions in the input vector.

Add the item *elem*.

Continue with the elements of the input vector from index n to the end.

We need to modify the code in *impRemove* in two ways: add *elem* after the first **for** loop, and start the second **for** loop with n (rather than $n + 1$).

```
function INSERT(n,v,elem)
{
  var temp in Vector of X
  var i in Int
  for (i <-- 1 to n-1)
  {
    temp.addLast(v.at(i))
  }
  temp.addLast(elem)           // elem is added at position n
  for (i <-- n to v.size())
  {
    temp.addLast(v.at(i))
  }
  return temp
}
```

In Unit 3, we mention the binary operation of concatenation of sequences. This process combines two sequences into a single sequence, by writing the members of the first sequence followed by those of the second sequence, without changing their order. (Note that if either of the sequences is empty, then the resulting sequence is just a copy of the other.) We now look at a function that performs this process on vectors.

function *JOINVECTORS*(*u* in Vector of X, *v* in Vector of X) **return** in Vector of X

pre true.

post The returned vector is the concatenation of the two input vectors in the order *u* then *v*.

Activity 2.6

Let *u* be the vector [1, 2, 3, 99], *v* the vector [1, 2] and *w* the empty vector. What are:

- (a) *JOINVECTORS*(*u*, *v*);
- (b) *JOINVECTORS*(*u*, *w*);
- (c) *JOINVECTORS*(*v*, *u*)?

Discussion

- (a) `[1, 2, 3, 99, 1, 2]`.
- (b) `[1, 2, 3, 99]`.
- (c) Note that the inputs are in the order v then u . The returned value is `[1, 2, 1, 2, 3, 99]`. This is a different sequence from that in (a), illustrating that the order in which two sequences are concatenated (u then v , or v then u) matters.

Activity 2.7

Give an implementation of *JOINVECTORS*.

Discussion To form *JOINVECTORS*(u, v), we can start by setting a variable vector *temp* to a copy of u . Then add the elements of v at the end of *temp*, one at a time.

```
function JOINVECTORS(u,v)
{
  var temp in Vector of X
  var i in Int
  temp <-- u.clone()
  for (i <-- 1 to v.size())
  {
    temp.addLast(v.at(i))
  }
  return temp
}
```

2.2 Sorting a sequence

Many tasks on sequences can be performed much more efficiently if the sequence is sorted. By “sorted”, we mean that the items appear in the sequence in accordance with some ordering of the data items. Think for a moment about the task of finding a word in a dictionary. A dictionary stores words in a particular way, first those starting with ‘a’, then those starting with ‘b’, and so on. If the words appeared in the dictionary in random order, would you be able to find a particular word?

We look now at the task of taking a sequence of items, and sorting it so that its items appear in an organised way (as in a dictionary). To be able to sort the items in a sequence, we need to have an ordering relationship for the type of items in the sequence. For example, integers are ordered by their size. Characters can be ordered by referring to their ASCII codes. Such ordering relationships on data items can be exploited in code if we have corresponding comparison operations available (or methods with an equivalent effect). For example, we have comparisons such as `<` and `<=` available in the type **Int**, based on the ordering of integers by their size. Comparisons operations are also available in **Char**, based on the ASCII codes of the characters.

We shall refer to a sequence of integers as **sorted** if the integers appear in the sequence in increasing order. That is, the smallest integer is at the front of the sequence and the largest is at the back, and every integer in the sequence is smaller than (or equal to) the next integer in the sequence.

So, for example, $[33, 2, 6, 2, 9, 8]$ is not sorted but $[2, 2, 6, 8, 9, 33]$ is sorted. The second of these sequences contains the same items as the first, but in the second they are stored in increasing order of size.

We can sort sequences containing items of other types in a similar way. In particular, for sequences of characters we sort by reference to their ASCII codes. For example, the sequence $['a', 'n', 'y']$ is sorted, but $['q', 'w', 'e']$ is not. To see why, we can refer to the usual alphabetical ordering of letter characters, which for lower-case letters corresponds to the ordering by ASCII codes. In the first sequence, the letters 'a', 'n' and 'y' do appear in alphabetical order. In the second, 'e' should come before 'q'.

Recall that the comparison type functions on **Char**, such as $<$, order characters by reference to their ASCII codes.

Activity 2.8

Which of the following sequences is sorted? For those that are not, give sequences that contain the same items and are sorted.

- (a) $[2, 7, 9, 9, 12, 150]$
- (b) $[5, 7, -9, 12, -16]$
- (c) $[1, 3, 1, 4, 1, 5, 3]$
- (d) $['s', 'o', 'r', 't', 'e', 'd']$
- (e) $['S', 'o', 'r', 't', 'e', 'd']$.

Discussion Only the sequence in (a) is sorted. In each case, a sorted sequence containing the same items is given below.

- (b) $[-16, -9, 5, 7, 12]$. (Remember that any negative integer is less than any positive integer, and -16 is less than -9 .)
- (c) $[1, 1, 1, 3, 3, 4, 5]$. (Remember that a repeated item must appear in the sorted version the same number of times as in the original sequence.)
- (d) $['d', 'e', 'o', 'r', 's', 't']$.
- (e) $['S', 'd', 'e', 'o', 'r', 't']$ (Remember that we sort by reference to the ASCII codes of the characters, and any upper-case letter has an ASCII code smaller than that of any lower-case letter.)

We now consider how the process of sorting a sequence can be achieved using methods of the class **Vector**. The topic of sorting is a large one, and we shall only touch on it here. We shall describe one implementation of the sorting process. This is not an “efficient” implementation, in that it will take longer to execute than other approaches to the problem, but it is an approach that is relatively easy to express in code. We shall only consider sorting a sequence of integers. The same approach can be used for sequences holding other types of data, so long as the data type has a method or type operation of comparison.

We first give a specification of a function that sorts a vector of integers. There is no need for a precondition other than **true**; any vector of integers can be sorted. (We regard the empty vector as already sorted.)

function *SORT*(*v* in Vector of Int) return in Vector of Int

pre true.

post The returned vector contains exactly the same items as *v*, but the elements appear in ascending order.

The approach on which our implementation of *SORT* is based is known as **bubble sort**. This approach has as its basis the comparison of adjacent pairs of numbers in the vector being sorted. The position of these numbers in the vector is changed, or not, according to their sizes.

A bubble sort is executed by repeatedly passing along the vector from the front to the back comparing adjacent elements, and exchanging them if necessary. After the first pass, the largest element will be in the correct position at the back of the vector. The second pass will then place the second largest element in its correct position. This continues until the final pass compares only the first two elements in the vector.

Bubble sort is really a strategy for sorting, rather than a specific expression of the strategy in code. The term **algorithm** is often used to mean a general strategy for approaching a particular problem. Bubble sort is an algorithm for sorting. We give below an implementation *impByBubbleSort* of the function *SORT*, which is based on the bubble sort algorithm.

```
function SORT(v)
{
    // impByBubbleSort
    var temp in Vector of Int
    var i in Int
    var j in Int
    var store in Int
    temp <-- v.clone()
    for (i <-- 1 to v.size()-1)      //outer i loop
    {
        for (j <-- 1 to v.size()-i)  //inner j loop
        {
            if (temp.at(j) > temp.at(j+1)) // (1)
            /* At (1) we compare items
               then the code below swaps them if necessary. */
            {
                store <-- temp.at(j)
                temp.setAt(j,temp.at(j+1))
                temp.setAt(j+1,store)
            }
        }
        //end of j loop
    }
    //end of i loop
    return temp
}
```

In the inner loop (with loop variable *j*), we work through the vector *temp*, starting at the left, and examining adjacent elements. If the element on the left of the pair is larger, then the two elements are exchanged. One effect of this is that the largest unsorted integer is moved into its correct position. In the outer loop (with loop variable *i*) the process is repeated for the entire input vector, so that each element is moved into position. The outer (*i*) loop works through the whole of the input vector, but the inner (*j*) loop needs only to work through the unsorted elements (of which there are fewer at each pass).

For the idea of “bubbling” to make sense, you need to think of the vector as being vertical. The smallest number should be at the top. At each exchange, if the smaller number is “below” then it “bubbles” upwards.

As a specific example, consider an input vector $[15, 14, 12, 16, 13, 11]$. This has size 6. The outer i loop will be executed 5 times (with i equal to 1, 2, 3, 4, and 5). The first execution of the inner loop, with $i = 1$, will take j from 1 to 5. The second inner loop (with $i = 2$) will take j from 1 to 4, the third will take j from 1 to 3, and so on. The table below shows the first two executions of the outer (i) loop for this input.

i	j	compare	exchange?	$temp$
—	—			[15, 14, 12, 16, 13, 11]
1	1	$15 > 14?$	Yes	[14, 15, 12, 16, 13, 11]
1	2	$15 > 12?$	Yes	[14, 12, 15, 16, 13, 11]
1	3	$15 > 16?$	No	[14, 12, 15, 16, 13, 11]
1	4	$16 > 13?$	Yes	[14, 12, 15, 13, 16, 11]
1	5	$16 > 11?$	Yes	[14, 12, 15, 13, 11, 16]
2	1	$14 > 12?$	Yes	[12, 14, 15, 13, 11, 16]
2	2	$14 > 15?$	No	[12, 14, 15, 13, 11, 16]
2	3	$15 > 13?$	Yes	[12, 14, 13, 15, 11, 16]
2	4	$15 > 11?$	Yes	[12, 14, 13, 11, 15, 16]

In this trace, we have shown in bold type the two elements that will be compared next.

Activity 2.9

Complete the trace started above of *impByBubbleSort* for the following input: $[15, 14, 12, 16, 13, 11]$.

Discussion Recall that $temp$ is in the state $[12, 14, 13, 11, 15, 16]$. The next execution of the outer loop has $i = 3$, and the inner loop is executed for j from 1 to 3.

i	j	compare	exchange?	$temp$
3	1	$12 > 14?$	No	[12, 14, 13, 11, 15, 16]
3	2	$14 > 13?$	Yes	[12, 13, 14, 11, 15, 16]
3	3	$14 > 11?$	Yes	[12, 13, 11, 14, 15, 16]
4	1	$12 > 13?$	No	[12, 13, 11, 14, 15, 16]
4	2	$13 > 11?$	Yes	[12, 11, 13, 14, 15, 16]
5	1	$12 > 11?$	Yes	[11, 12, 13, 14, 15, 16]

Execution of *impByBubbleSort* is now complete. The returned value is the vector $[11, 12, 13, 14, 15, 16]$. This is the input vector $[15, 14, 12, 16, 13, 11]$ correctly sorted.

Aside

We remarked above that bubble sort is not the most efficient algorithm available for sorting a vector. Efficiency is an idea that we will look at in some detail in Block 4, but it is of interest to give a brief indication of the issue here. Suppose that we count the number of times that the statement at line (1) is executed as *impByBubbleSort* is executed. Watching how this count grows as the input vector gets bigger provides an indication of how efficient the algorithm will be for large input vectors. For an input vector of size 6, this statement is executed a total of $5 + 4 + 3 + 2 + 1 = 15$ times. In general, for an input vector of size N , statement (1) will be executed a total of $\frac{1}{2}N(N - 1)$ times. For a large input vector (with large size N), the key feature in this count is the term $\frac{1}{2}N^2$. There are other sorting algorithms, for which the count of executed statements grows like $N \log N$ rather than like N^2 . For large values of N , this difference can be very significant.

Consider *impByBubbleSort* with an input vector [50, 12, 93, 20, 39]. With this input, the outer loop will execute four times. At the end of each of these executions, *temp* will be, in turn: [12, 50, 20, 39, 93]; then [12, 20, 39, 50, 93]; then [12, 20, 39, 50, 93] again; then [12, 20, 39, 50, 93] again. We could reduce the number of statements that are executed in such a case, by adding code to stop execution of the outer loop if the vector *temp* turns out to be sorted already. However, this will complicate the code (making it harder to ensure correctness), and it turns out not to provide a significant improvement in efficiency. To achieve that, we need to find an entirely different sorting strategy.

End of Aside

2.3 Extending the class Vector

We have discussed several functions in this section: *REMOVE*, *INSERT*, *SORT* and *JOINVECTORS*. Each of these would provide a useful addition to the class **Vector**. We look here at extending **Vector** to include corresponding methods. You have seen a number of examples where we add methods to a class by extension, such as **TurtlePlus extends Turtle** in Unit 5. The processes described earlier in the section are functions, and for each we need to adjust the specification and implementation to produce corresponding methods for the extended class. This remark is particularly relevant to *JOINVECTORS*.

The function *JOINVECTORS* is a binary operation on sequences. (Indeed, some programming languages provide types with a concatenation operation on sequences that uses infix notation, such as +.) A class cannot accommodate a binary operation as a method; at least, not directly. A method is always addressed to a receiver object, so one of the two input vectors to *JOINVECTORS* must be chosen to act as the receiver. We shall treat the first vector as the receiver. The specification of the method *join* that we shall use is given below.

method *join*(*v* in **Vector of X**) **state change**

pre **true.**

post Changes the state of the receiver by joining *v* to the end of the receiver.

Here, the parameter *v* is a vector. The method *join* will of course only be available to an object of a class extending **Vector** that has *join* as a method.

EXAMPLE 2.1

The class **VectorOne of X** extends **Vector of X** by adding the single method *join*. An implementation of **VectorOne of X** in a form suitable for the WorkPad is given overleaf. The code implementing *join* is based on the implementation of the function *JOINVECTORS* given in the comment on Activity 2.7. Here we do not need to use a local variable *temp*, since we want the method to change the state of the receiver object *this*.

(Note that, in the WorkPad, code to implement a parameterised class does not include “of X” in the class name nor in the **extends** statement.)

```

Class VectorOne

{
  this.join calls joinVectorOne
}

VectorOne extends Vector

method joinVectorOne(v)
{
  var i in Int
  var n in Int
  n <-- v.size()
  for (i <-- 1 to n)
  {
    this.addLast(v.at(i))
  }
}

```

We may call the method *join* in the form *v.join(v)* (where *v* is an object of type **VectorOne**). In that case, the use of a loop statement in the form **for** (*i* ← 1 **to** *v.size()*) causes an error, because the parameter is the same as the receiver, and the size of the receiver would change as the code implementing *join* is executed. This problem is avoided by the use of the local variable *n*. The code given above ensures that a call *v.join(v)* changes the state of *v* by adding a copy of the original state of *v* at the end of the vector (so changing the state from [1, 2, 3] to [1, 2, 3, 1, 2, 3], for example).

We mention the class **VectorOne** as a reminder of how we express in code a class implementation using extension. This class is not in the WorkPad library. That library does include the class **VectorPlus**, whose specification is given below. This class extends **Vector** by adding *join*, and also methods corresponding to the functions *REMOVE*, *INSERT* and *SORT*. The specification of *join* for **VectorPlus** is as given above. We give specifications of its other new methods after the class specification. Note that all the methods are state changers.

VectorPlus of X

Class specification

VectorPlus of X extends Vector of X

States, State notation and Default are as for **Vector**.

Methods

remove(*n* in Int) state change

insert(*n* in Int, *x* in X) state change

sort() state change

join(*v* in Vector of X) state change

plus all the methods of **Vector**.

We assume that **X** includes a comparison. For example, **X** might be **Int** or **Char**.

method *remove*(*n* in Int) **state change**

pre $1 \leq n$ and $n \leq$ the size of the receiver.

post Changes the state of the receiver by removing the element with index n .

method *insert*(*n* in Int, *x* in X) **state change**

pre $1 \leq n$ and $(n \leq 1 + \text{the size of the receiver})$.

post Changes the state of the receiver by inserting the element x so that it becomes the element with index n .

method *sort*() **state change**

pre true.

post Changes the state of the receiver by sorting its elements into ascending order.

Implementations of the methods of **VectorPlus** can be based on the implementations of the corresponding functions given earlier, as was *joinVectorOne* in Example 2.1. We omit the details.

Activity 2.10

Trace the states of the objects u and v as the code fragment below is executed.

```
var v in VectorPlus of Int
var u in VectorPlus of Int
u.setVector(4,78)
v.setVector(11,4,21,78,6)
v.remove(4)           //(1)
v.insert(3,21)        //(2)
v.join(u)             //(3)
v.sort()              //(4)
```

setVector is a method of **Vector**, and so is available to an object of type **VectorPlus**.

Discussion The states of the variables are traced below.

Comment	u	v
<i>u.setVector</i> (4,78)	[4, 78]	[]
<i>v.setVector</i> (11,4,21,78,6)	[4, 78]	[11, 4, 21, 78, 6]
<i>v.remove</i> (4)	[4, 78]	[11, 4, 21, 6]
<i>v.insert</i> (3,21)	[4, 78]	[11, 4, 21, 21, 6]
<i>v.join</i> (u)	[4, 78]	[11, 4, 21, 21, 6, 4, 78]
<i>v.sort</i> ()	[4, 78]	[4, 4, 6, 11, 21, 21, 78]

At (1), remember that the parameter in *remove* is the index position of the item to be removed. So it is the fourth item in v (the number 78) that is deleted (not the number 4). At (2), the second parameter (the number 21) is inserted so that its index is the first parameter, 3. All the other numbers in v remain, in the same order as before. (Those with positions after 3 now have their index position increased by 1, of course.) At (3), the state of v , the receiver, is changed by adding the elements of the parameter, u , at the end. (The state of u does not change.) At (4), the elements of v are sorted into increasing order. Remember that the resulting state should be of the same size as the previous state, with elements repeated the appropriate number of times.

It is important to note that all the added methods of this extended class are state changers, so a statement such as the following is not valid.

```
 $u.join(v.remove(3))$ 
```

The parameter for *join* must be a value (a vector), and *v.remove(3)* is *not* a value. It is an instruction to change the state of *v*. If you want to change the state of *v* by deleting its third element, and then add the resulting vector to the end of *u*, this can be achieved by the two statements below.

```
v.remove(3)
u.join(v)
```

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: sorted (sequence); algorithm.
- ☐ Understand the specifications of the functions on vectors introduced in the section: *DELETE*, *INSERT*, *JOINVECTORS* and *SORT*.
- ☐ Give implementations of functions with **Vector** in their signature.
- ☐ Understand the specification of the class **VectorPlus of X**, and trace the effect of code involving the methods of this class.
- ☐ Be aware of the difference between functions with **Vector** in their signature, and the methods of the class **VectorPlus** that extends **Vector**.
- ☐ For a particular input vector, trace the execution of the implementation *impByBubbleSort* of the bubble sort algorithm.

Exercises on Section 2

Exercise 2.1

A specification is given below of a function *BACK*. Give an implementation of *BACK*.

function *BACK*(*n* in Int, *v* in Vector of X) **return** in Vector of X

pre $1 \leq n$ and $n \leq$ the size of *v*.

post The returned value is a vector consisting of the last *n* items from *v*.

For example, *BACK*(4, [21, 6, 14, 3, 12, -21, 11]) = [3, 12, -21, 11].

Exercise 2.2

Suppose that *impByBubbleSort* is applied to the input vector [7, 4, 2, 3, 10, 9, 1]. What will be the state of *temp* after each execution of the outer loop? What value will be returned?

Exercise 2.3

Trace the states of the variables as the code fragment below is executed.

```
var vecOne in VectorPlus of Int
var vecTwo in VectorPlus of Int
var item in Int
vecOne.setVector(11,4,21,78,6)      //(1)
vecTwo <-- vecOne.clone()           //(2)
item <-- vecOne.at(3)
vecOne.remove(3)
vecOne.sort()
vecTwo.insert(2,item)
vecOne.join(vecTwo)
```

Exercise 2.4

A new class **VectorTwo of X** extends **Vector of X** by adding the single method *delall*, whose specification is given below. Give an implementation of **VectorTwo of X** in a form suitable for the WorkPad.

method *delall*(*item* in X) **return** in **VectorTwo of X**

pre true.

post The returned value is formed from the receiver by removing all occurrences of *item*. (If *item* does not occur in the receiver then the returned value is just the current state of the receiver.)

For example, for an object *v* in the state [2, 7, 3, 7, 2, 11], *v.delall*(7) will return the vector [2, 3, 2, 11].

3 The class *mString*

The second class we consider in this unit is **mString**. This is another important collection class whose form of data is a sequence, but it differs from **Vector of X** in a number of significant ways.

- **mString** is a highly specialised class. Its objects hold only characters. (It is not a parameterised class.)
- Except for *setString*, the methods all return a value; that is, there are no other state changing methods.
- Consequently, to change the state of an object of type **mString**, we must use assignment.

The state of an object of type **mString** is a sequence of characters. We introduced the terminology **string** for a sequence of characters in Unit 1. For convenience, we shall usually refer to an object of type **mString** simply as a string. In Unit 1, we also introduced a shorthand notation for strings, using double inverted commas. We shall use that here. Rather than writing ['a','b','c'], we just write "abc". In particular, the state of an object of type **mString** will be shown using double inverted commas.

The characters in a string are indexed like the elements in a vector, so that in "abc":

'a' has index 1; 'b' has index 2; 'c' has index 3.

3.1 The specification of the class *mString*

The specification of the class **mString** is given overleaf, followed by specifications of each of its methods. Some of the method specifications are quite complicated. We shall explore the effect of the methods further after giving their specifications.

mString	Class specification
<i>States</i>	A sequence of characters (a string).
<i>State notation</i>	The state is shown using the double inverted comma notation.
<i>Default</i>	The empty string (written "").
<i>Methods</i>	
	<i>setString(listOfChar)</i> state change
	<i>length()</i> return in Int
	<i>charAt(i in Int)</i> return in Char
	<i>substring(n in Int, m in Int)</i> return in mString
	<i>substr(n in Int, len in Int)</i> return in mString
	<i>charAppend(c in Char)</i> return in mString
	<i>concat(s in mString)</i> return in mString
	<i>equal(s in mString)</i> return in Bool
	<i>hashCode()</i> return in Int

Note that all methods of **mString** (other than *setString*) return a value and do not change the state of the receiver.

Programming languages are likely to have their own version of a string class. Do not expect these to be identical to our **mString** class, or to each other. In some programming languages, the string class does include state changing methods.

The default state of an object of type **mString** is the empty string. After a statement **var s in mString** the state of *s* is an empty sequence of characters, written as "". As with **Vector**, **mString** has a method to set an object to a particular value. The code below will declare an object *s* of type **mString**, and then set its state to "abc".

```
var s in mString
s.setString("abc")
```

Notice that the parameter for *setString* is given using the double quotes notation for a string. The specification of this method is given below.

method *setString(listOfChar)* **state change**

pre true.

post The state of the receiver becomes the sequence of characters given as *listOfChar*.

In code, the parameter *listOfChar* is entered as follows. Start with a double inverted comma, then give the characters without any separators, then another double inverted comma. Everything entered between the two double inverted commas, including spaces and commas, will be treated literally as characters of the string. Suitable values for the parameter *listOfChar* would include: "abc", "this is a string" and "" (the empty string).

In writing code, we need to be careful to distinguish between a variable of type **mString**, and a list of characters such as "qwerty" given in the double inverted comma notation. We use this double inverted comma notation to give the state of a variable of type **mString**, but do not think that "qwerty" is actually a literal of that type. Suppose that *s* is a variable of type **mString**. We cannot, for example, use an assignment *s* ← "qwerty". In choosing appropriate parameters for methods, we need to distinguish between a list of characters, such as "qwerty", and a variable of type **mString**. For *setString*, the parameter is a list of characters. For each of *concat* and *equal* (specified below), the parameter is of type **mString**.

The empty string is shown as two double inverted commas with no space between them. Since the space character is in **Char**, the string " " (with a space) is not the same as the empty string. In text, we may give this as "□" to make clear that there is a space character in the string. We cannot use the notation "□" in code, of course.

We may refer to either an explicit list of characters such as "qwerty", or an object of type **mString**, as a string. This is a small abuse of terminology. In code, we need to distinguish between these two ideas of a string.

We now look at the other methods of **mString**.

method *length*() **return in** **Int**

pre **true**.

post The returned value is the number of characters in the receiver.

method *charAt*(*n in Int*) **return in** **Char**

pre $1 \leq n$ and $n \leq$ the length of the receiver.

post The returned value is the character with index *n* in the receiver.

The precondition of *charAt* ensures that *n* is a valid position in the string.

A **substring** of a string is a continuous run of characters taken from that string. For example “wert”, “tyu” and “q” are all substrings of the string “qwertyu”. Each of the next two methods selects a substring from the receiver string, but they do this using different parameters. In each case, the precondition ensures that we do not look outside the receiver string when selecting the characters for the substring. We shall look more carefully at these methods later in the section.

method *substring*(*n in Int*, *m in Int*) **return in** **mString**

pre $1 \leq n$ and $n \leq m$ and $m \leq$ the length of the receiver.

post The returned value is a string consisting of a substring of characters taken from the receiver. This substring starts with the character at index position *n* and finishes with the character at index position *m*, including each of these.

method *substr*(*n in Int*, *len in Int*) **return in** **mString**

pre $1 \leq n$ and $n \leq$ the length of the receiver and

$0 \leq len$ and $(n + len - 1) \leq$ the length of the receiver.

post The returned value is a string consisting of a substring of characters taken from the receiver. This substring has exactly *len* characters, and starts with the character at the index position *n* in the receiver. If *len* is 0, the returned string is empty.

method *charAppend*(*c in Char*) **return in** **mString**

pre **true**.

post The returned value is a string consisting of the receiver with the character *c* added to the end.

The next method, *concat*, forms the returned string by concatenation, which (as for sequences in general) means combining the two sequences of characters into a single string. The method puts the parameter string after the receiver string.

method *concat*(*s in mString*) **return in** **mString**

pre **true**.

post The returned value is a string that has the string *s* concatenated to the end of the receiver.

Note that the parameter for *concat* must be another object of type **mString**, not a list of characters written using the inverted comma notation. The same comment applies to the method *equal*, below. This tests whether the receiver string is identical to the parameter string.

method *equal*(*s in mString*) **return in** **Bool**

pre **true**.

post The returned value is **true** if the receiver and the string *s* are identical (that is, have the same length and contain the same characters in the same order), and is **false** otherwise.

The final method, *hashCode*, will be used in the next section, and we leave details of how the returned integer is calculated until then.

method *hashCode*() **return in** **Int**

pre **true**.

post The returned value is an integer associated with the first three characters of the receiver.

Notice that there is only one method that changes the state of the receiver, namely *setString*, which is used to set the state to a given list of characters. All the other methods return a value. In particular there are a number of methods that return a new string. In each case, the returned string will need to be captured by assignment to a variable of type **mString**.

For instance, if we concatenate two strings *sOne* and *sTwo*, the result of applying the method *concat* will be a third string. If this is stored in another variable, as in the fragment below, then the state of neither *sOne* nor *sTwo* will be changed. (After this fragment, the states are: “abc” for *sOne*; “efg” for *sTwo*; “abcefg” for *sThree*.)

```
var sOne in mString
sOne.setString("abc")
var sTwo in mString
sTwo.setString("efg")
var sThree in mString
sThree <-- sOne.concat(sTwo)
```

3.2 User code involving the class **mString**

We now look at some examples of the effect of the methods of **mString**.

Activity 3.1

Suppose that *s*, *t* and *u* are objects of type **mString**, and their states are: for *s*, “abcde”; for *t*, “abc” and for *u*, “abcd”. What value will be returned by each of the following?

- (a) *s.length*()
- (b) *s.charAt*(2)
- (c) *s.charAt*(10)
- (d) *s.charAppend*(‘Q’)
- (e) *s.concat*(*t*)
- (f) *s.equal*(*u*)

Discussion

- The returned value is 5 (the number of characters in the string “abcde”).
- The returned value is ‘b’. Note that this is a character, namely the character with index 2 in the string “abcde”. (Remember that index positions are counted from the left end of the string, starting with 1.)
- This method call is invalid, because the inputs do not satisfy the precondition of the method *charAt*. The length of “abcde” is 5, and for these inputs the precondition of *charAt* requires that $10 \leq 5$, which is **false**. You should always take care to check and respect the precondition for any method or function.
- The returned value is “abcdeQ”. The method *charAppend* adds the character supplied as a parameter (‘Q’ here) to the end of the receiver string (“abcde” here).
- The returned value is “abcdeabc”. The method *concat* adds the string supplied as a parameter to the end of the receiver string.
- The returned value is **false**. The method *equal* will only return **true** if the parameter string is identical to the receiver string in every respect. They must have the same length, and contain the same characters in the same order. The strings “abced” and “abcde” are not identical, since the order of the last two characters is different.

The class **mString** has two different methods for extracting a substring from the receiver string. The method *substring* uses absolute positions in the string. A method call of the form *substring*(*firstPos*, *lastPos*) extracts from the receiver string the substring that starts at index position *firstPos* and ends at the index position *lastPos*.

For example, suppose that *s* is a variable of type **mString** in the state “There’s gold in them thar hills”, and we want to extract from it the string “gold”. To do this, first determine the index positions in *s* of each of the ‘g’ and the ‘d’ in “gold”. In doing this, we need to be careful to count each character in the string separately, including space characters and the apostrophe, as indicated in Figure 3.1. The figure shows that ‘g’ has index 9 and ‘d’ has index 12. So a call *s.substring*(9,12) will return a string in the state “gold”. (This returned value can then be assigned to a variable of type **mString**.)

	<i>substring</i> (9, 12)																
	↓								↓								
<i>index</i>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	...
<i>s</i>	T	h	e	r	e	'	s		g	o	l	d		i	n		...

Figure 3.1

The method *substr* extracts a substring in a different way. It again has two parameters, one of which is the index of the first character of the required substring. The second parameter of *substr* is the size of the required substring. We can use *substr* to perform the same task, of extracting “gold” from “There’s gold in them thar hills”. But for *substr*, the second parameter will be different.

Most programming languages will have a class handling strings that is similar to the class **mString**, but not identical. Methods equivalent to either or both of the **mString** methods for extracting a substring may be available.

Activity 3.2

Write a code fragment that: declares variables *s* and *t* of type **mString**; uses *setString* to set *s* to “There’s gold in them thar hills”; then uses the method *substr* to extract “gold” from *s*, and assigns that value to *t*.

Discussion A suitable fragment is given below. (Note that the target substring “gold” contains 4 characters.)

```
var s in mString
var t in mString
s.setString("There's gold in them thar hills")
t <-- s.substr(9,4)
```

Each of the methods *substring* and *substr* has a precondition. These ensure that when picking out the substring corresponding to the method call, one does not look beyond the ends of the receiver string. When calling these methods, it is important to bear these preconditions in mind.

Activity 3.3

Suppose that a variable *s* of type **mString** is in the state “abcde”. What value, if any, will be returned by each of the following?

- | | |
|---|--|
| (a) <i>s.substring</i> (1, <i>s.length</i> ()) | (b) <i>s.substring</i> (0, 3) |
| (c) <i>s.substring</i> (2, 6) | (d) <i>s.substring</i> (3, 2) |
| (e) <i>s.substring</i> (18, 20) | (f) <i>s.substring</i> (2, 2) |
| (g) <i>s.substr</i> (1, 1) | (h) <i>s.substr</i> (3, 2) |
| (i) <i>s.substr</i> (18, 2) | (j) <i>s.substr</i> (1, <i>s.length</i> ()) |

Discussion Many of these method calls fail the preconditions.

- (a) This call is valid and returns the value “abcde”.
- (b) The first parameter of *substring* must be ≥ 1 , and this is not true of 0. The precondition is not satisfied, and this call is invalid.
- (c) The second parameter of *substring* must be $\leq$ the length of the receiver string, which is 5 in this case. Now $6 \leq 5$ is not true, so the precondition is not satisfied, and this call is invalid.
- (d) The first parameter of *substring* must be $\leq$ the second parameter. That is not true in this case, since $3 \leq 2$ is false. So the precondition is not satisfied, and this call is invalid.
- (e) Both the parameters here are larger than the size of the receiver string. So the precondition is not satisfied, and this call is invalid.
- (f) This call is valid and returns the string “b”.
- (g) This call is valid. It returns the substring of “abcde” starting at index position 1, and with length 1; that is, “a”.
- (h) This call is valid. It returns the substring of “abcde” starting at index position 3, and with length 2; that is, “cd”.
- (i) The first parameter is larger than the length of the string. So the precondition is not satisfied, and this call is invalid.
- (j) This call is valid and returns “abcde”.

Note that “b” is a string, and is not the same as the character ‘b’.

The methods for extracting substrings, together with concatenation, provide powerful tools for building new strings from old. For example, suppose that *s* is a variable of type **mString** in the state “one two”, and we wish to reverse the order of the two words (to change the state to “two one”. (Note that there is one space between the words in each of these strings.) The code fragment below will do that.

```
var t in mString
t <-- s.substring(5,7)           //(1)
t <-- t.charAppend(' ')         //(2)
s <-- t.concat(s.substring(1,3)) //(3)
```

In (2), a space character has been entered between the single inverted commas by pressing the space bar.

In “one two”, “one” is the substring from index 1 to 3 and “two” is the substring from index 5 to 7. We have built up the required string by:

extracting the substring “two” (statement (1));
 appending a space character at the end of “two” (statement (2));
 concatenating the substring “one” after this (statement (3)).

Activity 3.4

Suppose that *s* and *t* are variables of type **mString**, with *s* in the state “The□rush” and *t* in the state “gold”. Write a code fragment that inserts *t* into *s* at the appropriate place, so as to change the state of *s* to “The□gold□rush”. (Do not change the state of *t*.)

We have used □ to show space characters in these strings.

Discussion We first concatenate at the end of *t* the substring “□rush” taken from *s*, and store the resulting value in a new variable *temp* (to avoid changing the state of *t*). We can then change the state of *s* to its substring “The□”. Finally, concatenate *temp* at the end of *s*.

```
var temp in mString
temp <-- t.concat(s.substr(4,5))
s <-- s.substr(1,4)
s <-- s.concat(temp)
```

3.3 Functions on mString

We look now at some functions that perform a particular specialist task on strings. For instance, consider the function *TRIMFRONT*, specified below.

function *TRIMFRONT*(*s* in **mString**) **return** in **mString**

pre *s* contains at least one character that is not a space character.

post The returned value is a string formed from *s* by removing any space characters that it may have at the front.

For example, if *s* is “□□□□This□is□a□test□string.”, then the returned value is the string “This□is□a□test□string.” with the four spaces at the front of the string removed.

To implement *TRIMFRONT*, we can first find the count of space characters at the start of the string. This task is most naturally done using a **while** loop.

The class **mString** has a relatively small number of general purpose methods.

Most programming languages will have string classes with more methods than **mString**, including methods that perform specialist tasks like this.

If we want a class with such methods, we can extend **mString** by supplementing it with additional methods.

Activity 3.5

Use a **while** loop to give an implementation of the function specified below.

function *FRONTCOUNT*(*s* in **mString**) **return** in **Int**

pre *s* contains at least one character that is not a space character.

post The returned value is the number of space characters at the front of the string *s*.

Discussion Starting at the front of the input string, we can examine its characters in turn until we arrive at one that is not a space character. There may be no spaces at all, so the variable giving the count should start at 0.

```
function FRONTCOUNT(s)
{
  var count in Int
  count <-- 0
  while (s.charAt(count+1) == ' ')
  {
    count <-- count + 1
  }
  return count
}
```

In the **while** condition used above the space character is entered literally, by typing a space character enclosed between single inverted commas. Since it is not easy to check your code to see whether this is correctly entered, you may prefer to write the condition as *ASC(s.charAt(count + 1)) == 32*. The precondition ensures that the **while** condition will become **false** before we reach the end of *s*.

Activity 3.6

Give an implementation of the function *TRIMFRONT*. Assume that the function *FRONTCOUNT* is available (as well as methods of **mString**).

Discussion We want to return the appropriate substring of the input string *s*.

```
function TRIMFRONT(s)
{
  return s.substring(FRONTCOUNT(s)+1, s.length())
}
```

Activity 3.7

Let t be an object of type **mString** in the state “□□□□Test□□□□”. What value would be returned as each of: *FRONTCOUNT*(t); *TRIMFRONT*(t)?

Discussion The returned values will be: *FRONTCOUNT*(t) = 5 and *TRIMFRONT*(t) = “Test□□□□”. Note that there are still four space characters at the end of this returned string.

Some tasks require us to work our way through a string, processing each character in turn and storing the processed character in a new string. For example, consider the function *TOUPPER*, specified below.

function *TOUPPER*(s in **mString**) **return** in **mString**

pre true.

post The returned string is the same as the string s , except that every lower-case letter character in s has been changed to the corresponding upper-case letter.

For example, *TOUPPER*(“aQ2!!+yT.”) = “AQ2!!+YT.”

In implementing *TOUPPER*, recall that to change a lower-case letter into the corresponding upper-case letter, we find the ASCII code of the character, subtract 32 from this integer, then change it back to a character using *CHR*.

See Unit 3, p16.

Activity 3.8

Give an implementation of *TOUPPER*. (Start with an empty variable string, say *temp*. Work through the input string, character by character. If a character is not a lower-case letter, put it into *temp* unchanged. If it is a lower-case letter, put the corresponding upper-case letter into *temp*.)

Discussion An implementation is given below. We store the i th character of the string s in c , check at (1) whether this is a lower-case letter, and if it is then change it to its upper-case equivalent at (2).

```
function TOUPPER(s)
{
  var temp in mString
  var i in Int
  var c in Char
  for (i <-- 1 to s.length())
  {
    c <-- s.charAt(i)
    if ((c >= 'a') /\ (c <= 'z')) then    //(1)
    {
      c <-- CHR(ASC(c) - 32)              //(2)
    }
    temp <-- temp.charAppend(c)
  }
  return temp
}
```

Word processing software will contain many routines for handling strings. One example is to locate a target string within a larger string. We now specify a function relating to this task, which returns an index position showing where the target string is to be found in the larger string.

function *MATCH*(*s* in *mString*, *target* in *mString*) **return** in *Int*

pre *target* is not empty.

post Returns the index position of the start of the first occurrence of *target* in *s* as a substring. If *target* does not occur as a substring of *s* then 0 is returned.

The function *MATCH* is illustrated in Figure 3.2, which shows that if *s* is “This is a test string.” and *target* is “test”, then the value returned by *MATCH*(*s*,*target*) will be 11.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	<i>index</i>
T	h	i	s		i	s		a		t	e	s	t		s	t	r	i	n	g	.	<i>s</i>
										↑	↑	↑	↑									<i>target</i>
										t	e	s	t									

Figure 3.2

Activity 3.9

What value will be returned by *MATCH*(*s*, *target*) if *s* is “This is a test string.” and *target* is: (a) “is”; (b) “that”?

Discussion (a) If *s* is “is”, then the value returned will be 3. This is the index position of the start of the first occurrence of “is” as a substring of *s*, which is in the word “This”. (b) If *s* is “that”, then 0 will be returned, since “that” is not a substring of the given string *s*.

We will ask you to implement this function *MATCH* in Exercise 3.3.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: substring, and other terminology associated with the class *mString*, including the names of its methods.
- ☐ Use the methods of the class *mString*, including an awareness of their preconditions.
- ☐ Give implementations of functions with *mString* in their signature.

Exercises on Section 3

Exercise 3.1

A specification is given below of a function *SECWORD* and the first character in *s* is not a space character. For example, *SECWORD*(“This is a cat.”) = “is”. Give an implementation of *SECWORD*.

function *SECWORD*(*s* in *mString*) **return** in *mString*

pre *s* contains at least two space characters and the first character in *s* is not a space character.

post The returned value is a string consisting of the substring of *s* between (but not including) the first and second space characters in *s*. (If these space characters are adjacent, then the returned string is empty.)

Exercise 3.2

Let i be of type **Int**, $temp$ be of type **mString**, and s be of type **mString**, where $i \geq 1$ and s has length at least $i + 2$.

- (a) Using methods of **mString**, give a condition that the following holds. The character in s with index i is a comma, and the next two characters in s are both space characters.
- (b) Give a code fragment with the following effect.
 1. Set $temp$ to the substring of s consisting of the characters of s with index values from 1 to $i - 1$. Use a conditional statement that sets $temp$ to the empty string if i is 1.
 2. Append a comma at the end of $temp$.
 3. Append a space character at the end of $temp$.
 4. At the end of $temp$, concatenate the substring of s consisting of the characters with index values from $i + 3$ to the end of s . Use a conditional statement that does not change $temp$ if $i + 3$ is greater than the length of s .

Exercise 3.3

The specification of the function *MATCH* was given in the text. To implement *MATCH*(s , $target$), we can use the following strategy. The key idea is to use the method *substr* to compare the input string $target$ with the substring of s starting at i and of the appropriate length.

1. Work through the input string s using an integer variable i . This is the value that will be returned. Initialise i as 0.
2. Use a Boolean variable $found$ to indicate when the target string is located in s . Initialise $found$ as **false**.
3. Use a Boolean variable $done$ to indicate when a copy of $target$ cannot fit into the remainder of s . Initialise $done$ as **true** if $target$ is larger than s , and as **false** otherwise.
4. In a **while** loop that repeats so long as neither $found$ nor $done$ is **true**:
 - increase i by 1;
 - compare $target$ with the appropriate substring of s starting at i , and if $target$ is located set $found$ to **true**;
 - set $done$ to **true** if, when we start at index $i + 1$ in s and count forward the length of $target$, then we will go past the end of s .
5. If $target$ is never located, $found$ will still be **false** when the loop is left. If this is the case, use a conditional statement to set i to 0.
- (a) Using methods of **mString**, express each of the following conditions.
 - (i) The string $target$ is equal to the substring of s starting at i , and of the appropriate length.
 - (ii) If we start at index $i + 1$ in s and count forward the length of $target$, we go past the end of s .
- (b) Give an implementation of *MATCH*.

Pattern matching is an important part of text handling programs. This example illustrates that our relatively simple class **mString** has enough methods to perform this task.

4 Hash tables

Computers are frequently used for storing large amounts of data in a readily accessible manner. Databases are a very common form of data storage, and we shall discuss these in Block 3. This section will provide a brief introduction to some of the aspects that must be considered when storing large and increasing quantities of items within a collection object.

First, we shall consider the problem of searching a large data store for particular values. As an example, consider a dictionary. This will contain a collection of strings, which could be stored as an object of type **Vector of mString**. Real dictionaries will be very large, but for illustrative purposes, we will work with small examples. Here is a possible dictionary.

```
[“aardvark”, “lion”, “leopard”, “zebra”, “tiger”, “antelope”, “anteater”]
```

If the dictionary was being used for a spell checker in a word processing program, then the most common process to be performed would be to check whether or not a particular string lies in this dictionary. The class **Vector** has a method (*contains*) for doing that. However, it is of interest to consider how this process may be implemented.

We can start at the front of the vector, and examine each entry in turn, checking to see whether it is the desired string. For example, if we look for “cheetah” in the dictionary above, then we need to compare “cheetah” with each of the seven entries. We may need to check every entry in the dictionary before we either find the element we want, or know for certain that it is not there. For a large data store, this procedure will be rather long.

Now a feature of real dictionaries is that their entries are arranged in a systematic way. Using the terminology that we introduced in Section 2, a dictionary is a sorted vector. The sorting is based on an order relation for strings that is referred to as **lexicographic** order. Strings starting with ‘a’ are placed before strings starting with ‘b’, etc. This is not enough to order all strings though. A string starting “aa” comes before one starting “ab”; a string starting “cor” comes before one starting “cos”, and so on. Our example dictionary above would be arranged thus.

```
[“aardvark”, “anteater”, “antelope”, “leopard”, “lion”, “tiger”, “zebra”]
```

To search this sorted sequence of strings for “cheetah” we might proceed as follows.

- Start in the middle of the dictionary, which is “leopard” here.
- No match is made. Note that “cheetah” comes before “leopard” in lexicographic order. So look in the left half of the dictionary, which here is [“aardvark”, “anteater”, “antelope”], and apply the same process to that.
- The centre of this is “anteater”. Again no match is found. This time, “cheetah” comes after “anteater”. So look in the right half of the remaining dictionary. This just contains one item, [“antelope”].
- Since “cheetah” is not the same as “antelope”, we can conclude that “cheetah” is not in the dictionary.

We can use this process (which is an example of recursion) to determine whether or not a string lies in a large dictionary much more rapidly than by examining each item in turn. (When looking up a word in a physical

dictionary, we certainly do not start at the beginning and work through word by word! We do not replicate the “halving” strategy exactly, but that is closer to what one does.)

Sorting a vector enables us to search it more quickly. An unsorted vector needs to be searched item-by-item (with potentially every item being examined). This will involve more steps (potentially, many more), than a search based on the halving process described above. For a dictionary containing 7 items (as in the above example), the “halving” strategy requires us to examine 3 items at most, while searching item by item from the front requires us to examine 7 items at most. But for a larger dictionary, the contrast might be between 32 items and $2^{32} - 1$ items, that is 4294 967 295 items!

We now consider an alternative form of collection containing strings which also allows us to find an entry rapidly. We first associate an integer with each string that is to be stored. The string is then stored in a sequence, at the position in the sequence whose index is this integer.

This idea is not something that is suitable for human use, as it is based on a computer’s ability to do arithmetic very rapidly.

The number associated with a string is called its **hash code**, and the purpose of including the method *hashCode* in the class **mString** was to enable us to associate a hash code with a string. We shall discuss how the **mString** method *hashCode* works only for strings all of whose characters are either upper- or lower-case letters. Each letter character has an associated integer, as given in Table 4.1 below. Examine the first three characters in the string, and add together the numerical codes of the three characters as given in Table 4.1. (If the string has less than three characters, then add together the codes of all its characters.)

A = 1	K = 11	U = 21	e = 37	o = 47	y = 57
B = 2	L = 12	V = 22	f = 38	p = 48	z = 58
C = 3	M = 13	W = 23	g = 39	q = 49	
D = 4	N = 14	X = 24	h = 40	r = 50	
E = 5	O = 15	Y = 25	i = 41	s = 51	
F = 6	P = 16	Z = 26	j = 42	t = 52	
G = 7	Q = 17	a = 33	k = 43	u = 53	
H = 8	R = 18	b = 34	l = 44	v = 54	
I = 9	S = 19	c = 35	m = 45	w = 55	
J = 10	T = 20	d = 36	n = 46	x = 56	

Table 4.1

So, for example, to find the hash code of the string “aardvark”, add together the codes of ‘a’, ‘a’ and ‘r’, to get $33 + 33 + 50 = 116$; to get the hash code of “zebra”, add together the codes of ‘z’, ‘e’ and ‘b’, to get $58 + 37 + 34 = 129$; to get the hash code of “An”, add together the codes of ‘A’ and ‘n’ to get $1 + 46 = 47$.

Activity 4.1

Find the hash codes of each of the strings “lion”, “puma” and “tiger”.

Discussion We get:

for “lion”: $44 + 41 + 47 = 132$;
 for “puma” $48 + 53 + 45 = 146$;
 for “tiger”: $52 + 41 + 39 = 132$.

The smallest value returned by the method *hashCode* is for the string “A”, and is 1, while largest value returned is for “zzz”, and is 174. So a collection object that is to hold strings of letter characters based on their hash codes will need to have at least 174 places (one for each possible value of the hash code).

An important point to note is illustrated in Activity 4.1. Different strings may have the same hash code. Such an event is called a **collision**. We shall look at a way of dealing with collisions called **chaining**.

We call our collection a **hash table**. At each location in a hash table, we will have a number of strings stored. This number of strings may be none, one, or several. If we add an item to the collection, this will need to be added at the location appropriate to its hash code, which will increase the number of items stored there.

So a hash table collection is a sequence of locations. The size of this sequence is fixed (for our hashing procedure, 174 locations are needed). At each location a sequence of strings is stored. These sequences will vary in size, so need a type with dynamic growth. One might use the type **Array of Vector of mString** to describe a hash table. However, it is likely to be more appropriate to introduce a separate class **HashTable**, with its own methods specific to the uses of the class. The preceding discussion indicates a suitable type for an attribute to implement the form of data of a **HashTable** class.

We can reduce the number of collisions by increasing the complexity of the *hashCode* method, say by adding the codes of the first four letters in the string. This will increase the number of locations needed in the collection. Even if we do this, we will still find that collisions occur.

Activity 4.2

Suppose that the strings “aardvark”, “zebra”, “lion”, “puma” and “tiger” are entered into a hash table, where the entries in the table are of type **Vector of mString**. For each non-empty place in the table, give the hash code and the corresponding vector of strings.

Discussion We calculated the hash codes of these strings above (see Activity 4.1 and the preceding text). The hash table entries are given below.

hash code	vector
116	[“aardvark”]
129	[“zebra”]
132	[“lion”, “tiger”]
146	[“puma”]

We have not specified the order in which strings are entered when there is a collision, so it would be equally accurate to have [“tiger”, “lion”] for hash code 132.

A hash table provides an efficient method for storage, with a view to adding to, deleting from, and searching the data. However, a hash table is not very effective when it comes to iterating through the elements in order to apply some process to each element in turn. A hash table will be large, and may well have a number of positions containing empty vectors. A vector is a collection object better suited for such a purpose.

Hash tables need to be large enough to avoid too many collisions but small enough not to waste a lot of space. This depends on a judicious choice for the hashing process and the size chosen for the table. For instance, hashing the first three letters by multiplying them together rather than adding them would increase the size of the table needed, but would reduce the number of collisions.

In Unit 8 we will consider a quite different collection class, **BinTree of X**, that enables quick searches through large amounts of stored data based on the “halving” tactic described earlier.

In this brief discussion, we have considered only storage of strings. Now a dictionary is often used to store additional information. As an example, suppose we wish to store further information about each animal species relating to its colouration. We might store such combined information as a pair of strings, held as a vector. Each item in such a dictionary would be of type **Vector of mString**. Possible examples are given below:

```
[“lion”, “plain”], [“leopard”, “spotted”], [“zebra”, “striped”], [“tiger”, “striped”]
```

Each element in the store now carries two pieces of information. In many data stores each element (sometimes called a record in the store) will carry several pieces of information. Such data stores are discussed in Block 3.

When searching for information about “lion”, say, in such a store, it will be convenient if there is at most one entry relating to “lion”. If each element in the collection is uniquely identified by its first entry (the type of animal), then we would refer to this as a **key**. (Notice that the second entry in our example does not form a key. Different animals are described as “striped”.) As noted above, searching is more efficient if the items in the dictionary are sorted. We can use the key to do this sorting, which would arrange our examples above in a vector as below.

```
[ [“leopard”, “spotted”], [“lion”, “plain”], [“tiger”, “striped”], [“zebra”, “striped”] ]
```

When searching for an item in the dictionary, we would concentrate on the key. We could use the “halving” strategy we described above to speed up searching. Alternatively we can use hash codes as a basis for storage. We would use the hash code of the key to determine the appropriate storage location in the hash table.

In general, the fundamental form of data item stored in a hash table might be a string, or it might be anything, so long as we have a way of associating a hash code with a data item. There is a **HashTable of X** class in the WorkPad class library. This has methods: *put* (that adds a new entry to the hash table); *remove* (to delete an entry); *get* (to examine entries); *setTable* (that sets the size of a table); and *search* (to determine whether an item is present).

Objectives for Section 4

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: hash code, hash table, collision, lexicographic order (for strings).
- ☐ Calculate values returned by the method *hashTable* of **mString**.
- ☐ Understand how hash codes are used to place strings in a table and how collisions are dealt with.

Summary of Unit 6

1. An object of type **Vector of X** is a collection that holds data in a sequence. Each of the items in the sequence is of type **X**. We do not mix objects of different types within the same vector. However, **X** may be any type, and we say that **Vector of X** is a parameterised class. **Vector of X** has sufficient methods for objects of this type to have both random access (like an array) and dynamic growth (like a stack).
2. Processes not available as methods of **Vector** may be implemented as functions. Alternatively, one may extend the class **Vector** to incorporate additional processes as extra methods. We considered in particular: insertion, deletion, concatenation and sorting. A sequence of integers is said to be sorted if its members appear in increasing order. We can also sort sequences whose members are of other types, such as characters or strings.
3. The class **mString** is another sequence-based type, but is much more specialised than **Vector** in its use. An object of type **mString** is a sequence of characters. The methods of **mString** return values, so in code involving variables of this type, one needs to use assignment statements. The class **mString** has sufficient methods to implement certain functions associated with word processing, such as finding the location of a target string within a larger string (“find” a word in a document).
4. We briefly discussed searching in stored data, and hash tables.

Solutions to the exercises

Solution 1.1

(a) A trace is given below.

Comment	<i>v</i>	<i>i</i>	<i>v.at(i) == 99</i>	<i>found</i>
before loop	[1, 99, 3, 99, 5]	0		false
while condition is true	[1, 99, 3, 99, 5]	1	(1 == 99) is false	false
while condition is true	[1, 99, 3, 99, 5]	2	(99 == 99) is true	true
while condition is false				
<i>v.setAt(i, 100)</i>	[1, 100, 3, 99, 5]	2		true

After setting *v* to an initial state at (1), the code finds the first occurrence of 99 in *v*, and replaces this by 100. The final state of *v* is [1, 100, 3, 99, 5]; the final state of *i* is 2, and shows the index position of this first occurrence of 99 in the initial state of *v*; and the final state of *found* is **true**, showing that 99 has been found.

(b) If *v* is set to [1, 2, 3, 99, 5] at (1), then the code will have a similar effect to the trace in part (a). This time, the first occurrence of 99 in *v* is at index position 4. The final states of the variables will be: *v*, [1, 2, 3, 100, 5]; *i*, 4; *found*, **true**.

(c) If the vector *v* does not contain an entry that is 99, then the **while** condition will never become **false**. Continued execution of the body of the loop will eventually increase *i* to become larger than the size of *v*. Then *v.at(i)* will be called with a value of the parameter that fails the precondition of *at*. So the code will be invalid for any vector *v* that does not contain at least one entry that is 99.

Solution 1.2

An implementation is given below. We use a variable integer *soFar* giving the largest number found so far. Examine each number in the vector *v* in turn, starting at the left, and change *soFar* if the current integer in *v* is bigger. Initialise *soFar* as the first integer in *v*.

```
function MAX(v)
{
  var soFar in Int
  var i in Int
  soFar <-- v.at(1)
  for (i <-- 2 to v.size())
  {
    if (soFar < v.at(i)) then
    {
      soFar <-- v.at(i)
    }
  }
  return soFar
}
```

Solution 1.3

We will examine the items in *v* one at a time using *v.at(i)*, where *i* starts at 1. We need to stop the loop if we find an item larger than *target*. We also need to stop if we have examined all the items in *v*. We need to distinguish the case where no suitable item is found in *v*, since 0 must be returned in that case. To do this, we will use two Boolean variables: *found*, which will be set to **true** if the current item in *v* satisfies the condition *target* < *v.at(i)*; and *done*, which will be set to **true** if we reach the end of *v*.

```
function FIRSTOVER(target, v)
{
  var found in Bool
  var done in Bool
  var i in Int
  found <-- false
  done <-- false
  i <-- 0
  while (NOT(found) /\ NOT(done))
  {
    i <-- i + 1
    if (target < v.at(i)) then
    {
      found <-- true
    }
    if (i >= v.size()) then
    {
      done <-- true
    }
  }
  if (NOT(found)) then
  {
    i <-- 0
  }
  return i
}
```


Solution 2.1

Since we need n items taken from the back of v , it is convenient to start copying items from the back of v . The implementation given below works in that way. As i varies from 1 to n , so the value of $v.size() + 1 - i$ varies from $v.size()$ to $v.size() + 1 - n$. So the loop picks out n elements from v , starting with the last element (whose index is $v.size()$). To maintain the order of the items, each is added in turn to the front of $temp$.

One could just as well work the other way, starting at the item with index $v.size() - n + 1$, and adding items to the back of the vector variable.

```
function BACK(n,v)
{
  var temp in Vector of X
  var i in Int
  for (i <-- 1 to n)
  {
    temp.addFirst(v.at(v.size() + 1 - i))
  }
  return temp
}
```

Solution 2.2

The state of the vector $temp$ after each execution of the outer loop is given in the table below. Notice how in the first execution, the first item in the vector, 7, moves right until it encounters a larger number (10). Then 10 moves right. Since 10 is the largest number in the vector, it moves to the end.

In the second execution of the outer loop, the new first item, 4, moves right until it reaches 7. 7 does not move, since 9 is larger. Then 9 moves right.

The only item out of place is now 1. This is the smallest item in the vector. It “bubbles” one place left in each subsequent execution, until it gets to the first position.

The value returned by *impByBubbleSort* with input [7,4,2,3,10,9,1] is [1,2,3,4,7,9,10].

Comment	$temp$
Before first loop	[7, 4, 2, 3, 10, 9, 1]
After one execution	[4, 2, 3, 7, 9, 1, 10]
After two executions	[2, 3, 4, 7, 1, 9, 10]
After three executions	[2, 3, 4, 1, 7, 9, 10]
After four executions	[2, 3, 1, 4, 7, 9, 10]
After five executions	[2, 1, 3, 4, 7, 9, 10]
After six executions	[1, 2, 3, 4, 7, 9, 10]

Solution 2.3

The states are traced below.

Comment	$vecOne$	$vecTwo$	$item$
after (1)	[11, 4, 21, 78, 6]	[]	—
after (2)	[11, 4, 21, 78, 6]	[11, 4, 21, 78, 6]	—
$item \leftarrow vecOne.at(3)$	[11, 4, 21, 78, 6]	[11, 4, 21, 78, 6]	21
$vecOne.remove(3)$	[11, 4, 78, 6]	[11, 4, 21, 78, 6]	21
$vecOne.sort()$	[4, 6, 11, 78]	[11, 4, 21, 78, 6]	21
$vecTwo.insert(2,item)$	[4, 6, 11, 78]	[11, 21, 4, 21, 78, 6]	21
$vecOne.join(vecTwo)$	[4, 6, 11, 78, 11, 21, 4, 21, 78, 6]	[11, 21, 4, 21, 78, 6]	21

Solution 2.4

Suitable code is given below. Recall that when we give code implementing a user-defined class in the WorkPad, we do not include “of X” either in the class name or in the **extends** statement.

The method *delall* is implemented by *delallVectorTwo*. We copy the items we wish to keep to a temporary vector, which is returned. The specification of *delall* states that the returned value is of type **VectorTwo**, so the variable *temp* that is to be returned must be declared to be of type **VectorTwo** in the **var** statement.

```
Class VectorTwo

{
  this.delall calls delallVectorTwo
}

VectorTwo extends Vector

method delallVectorTwo(item)
{
  var temp in VectorTwo of X
  var i in Int
  for (i <-- 1 to this.size())
  {
    if (NOT(this.at(i) == item)) then
    {
      temp.addLast(this.at(i))
    }
  }
  return temp
}
```

Solution 3.1

An implementation is given below. We use **while** loops to find the first two space characters, and save the index of each, then extract the required substring of *s*. To avoid the use of literal space characters, we choose to test using the ASCII codes. If the two space characters occur together, then the call of *substring* at (2) would not satisfy its precondition. In that case, we just want to return an empty string, and before the conditional statement at (1) *out* will still be in the default state, which is an empty string. So we just use (1) to guard the call of *substring*; there is no need to change *out* if the condition in (1) is **false**.

```
function SECWORD(s)
{
  var out in mString
  var first in Int      // find index of first space
  var second in Int     // find index of second space
  first <-- 1
  while (NOT(ASC(s.charAt(first)) == 32))
  {
    first <-- first + 1
  }
  second <-- first + 1
  while (NOT(ASC(s.charAt(second)) == 32))
  {
    second <-- second + 1
  }
  if (NOT(second == first + 1)) then           //(1)
  {
    out <-- s.substring(first + 1, second - 1)  //(2)
  }
  return out
}
```

Solution 3.2

(a) We again test using the ASCII codes. The condition can be expressed as below.

$(ASC(s.charAt(i)) == 44) \wedge (ASC(s.charAt(i+1)) == 32) \wedge (ASC(s.charAt(i+2)) == 32)$

(b) A suitable code fragment is given below.

```
if (i == 1) then
{
  temp.setString("")
} // sets temp to an empty string if i is 1
else
{
  temp <-- s.substring(1, i - 1)
} // the rest of step 1.
temp <-- temp.charAppend(CHR(44)) // step (2)
temp <-- temp.charAppend(CHR(32)) // step (3)
if (i + 3 <= s.length()) then
{
  temp <-- temp.concat(s.substring(i + 3, s.length()))
} // step (4)
```

Solution 3.3

(a) (i) The substring of s that we want to compare with $target$ can be extracted as $s.substr(i, target.length())$. We can use the **mString** method *equal* to compare this with $target$. The required condition is

$target.equal(s.substr(i, target.length()))$

(ii) If we start at index $i + 1$, and count forward $target.length()$ places, we get to index $i + 1 + target.length() - 1$. (Remember that we count index position $i + 1$ at the start.) So the required condition is
 $i + target.length() > s.length()$.

(b) An implementation is given below, with some notes following the code.

```
function MATCH(s,target)
{
  var i in Int
  var found in Bool
  var done in Bool
  i <-- 0
  found <-- false // (1)
  if ( target.length() > s.length() ) then // (2)
    { done <-- true }
  else
    { done <-- false }
  while (NOT(found) /\ (NOT(done)) ) // (3)
  {
    i <-- i + 1 // (4)
    if (target.equal(s.substr(i, target.length()))) then // (5)
    {
      found <-- true // (6)
    }
    if ((i + target.length()) > s.length()) then // (7)
    {
      done <-- true
    }
  } // end of loop body
  if (NOT(found)) then // (8)
    { i <-- 0 } // (9)
  return i
}
```

(1) *found* will be set to **true** when the current value of i indicates the start of a substring of s that matches $target$. Set *found* to **false** initially, since no matching substring has yet been found.

(2) *done* will be set to **true** if, starting at index i , there are not enough characters in s to accommodate a copy of $target$. If $target$ is a longer string than s , then this will be true at the start. This conditional statement will set *done* to **true** if that is the case and to **false** otherwise.

(3) Repeat the **while** loop so long as two things are **true**: the $target$ substring has not been found, and there are enough characters in s from i to the end to fit in a copy of $target$.

(4) Increase i by 1 each time the loop is repeated.

(5) If the substring of s starting at index i and with the same length as $target$ is equal to $target$...

... then set *found* to **true** (6).

(7) Set *done* to **true** if we are too near to the end of s .

(8) If $target$ was never found in s ...

... then set i to 0 (9).

Index

- algorithm, 25
- bubble sort, 25
- casting, 11
- chaining, 44
- collection type, 4
- collision, 44
- constructor, 7
- dynamic growth, 4
- hash code, 43
- hash table, 44
- key, 45
- lexicographic (order for strings), 42
- mString**, 31
- random access, 4
- sorted sequence, 24
- string, 31
- substring, 33
- vector, 6
- Vector of X**, 7
- VectorPlus of X**, 28
- vector, back of, 6
- vector, first element of, 6
- vector, front of, 6
- vector, last element of, 6
- while** condition, 13
- while** loop, 13
- while** loop, body of, 13